

CS7CS3 Advanced Software Engineering Group Project

Functional and Technical Architecture

Project Name: *Response to City Disaster*

Group: 8

*mjaiswal@tcd.ie, mathankr@tcd.ie, courtinm@tcd.ie, fraiolim@tcd.ie,
taurov@tcd.ie, bharambn@tcd.ie, birarit@tcd.ie, kirthivs@tcd.ie,
jainpr@tcd.ie, taraneka@tcd.ie*

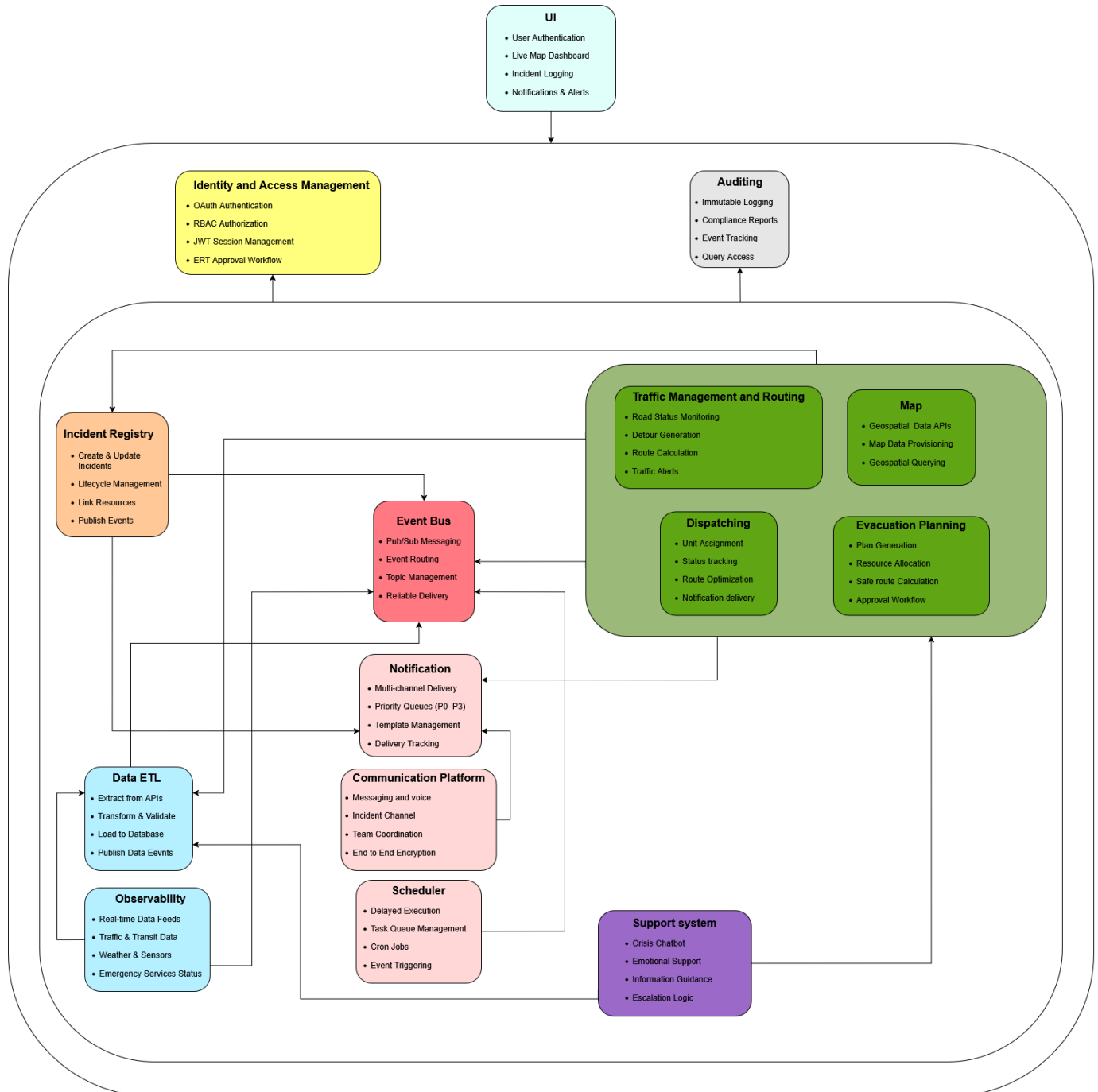
Contents

1. Functional Architecture	4
1.1. Diagram	4
1.2. Component Descriptions	5
Identity and Access Management (IAM)	5
Responsibilities	5
Event Bus	5
Scheduler	6
Notification	6
Data ETL	7
Observability	7
Map	8
Incident Registry	8
Communication Platform	9
Dispatching	9
Traffic Management and Routing	10
Evacuation Planning	11
Support System	11
Auditing	12
1.3. Component interactions	13
Identity Access Management	13
Manage and Process Real-time Data Sources	16
Log Disaster Event	19
Manage Evacuation Plan	20
Dispatch Emergency Unit	23
Re-Routing Public Traffic	25
View Live Disaster Map	26
Crisis Emotional Support and Information Bot	28
1.4. Component APIs	29
Identity and Access Management (IAM)	29
Event Bus	29
Scheduler	30
Notification	30
Data ETL	31
Observability	32
Map	36
Incident Registry	36
Communication Platform	37
Dispatching	38
Traffic Management	39
Evacuation Planning	40
Support System	40
Auditing	41

2. Technical Architecture	42
2.1. <i>Diagram</i>	42
2.2. <i>Quality of Service Technical Requirements</i>	44

1. Functional Architecture

1.1. Diagram



The four components Map, Traffic Management and Routing, Dispatching and Evacuation Planning are grouped within the same bounded context, reflecting their tight coupling and shared operational domain during disaster response operations.

1.2. Component Descriptions

Identity and Access Management (IAM)

The Identity and Access Management (IAM) component is the security foundation for all user interactions within the disaster management system. It provides comprehensive identity governance through robust user authentication, authorisation, and granular access control. By implementing industry-best security practices, such as Role-Based Access Control (RBAC) and OAuth-based login, the IAM ensures that only verified and appropriately authorised users can access specific system resources. This component coordinates security efforts across all other parts of the system to provide a secure and user-friendly identity lifecycle.

Responsibilities

- Provide distinct registration forms for end users and Emergency Response Team (ERT) members, securely collecting necessary information.
- Authenticate users using OAuth-based validation flows (e.g., via Google).
- Connect to and utilise external authentication systems, such as LDAP/Active Directory, for seamless authentication of organisational staff (e.g., ERT or Control Room personnel).
- Upon successful registration, place ERT accounts into a `PENDING_APPROVAL` status, requiring verification by a manager before full access is granted.
- Create and manage secure session tokens (such as JWTs), enforce session timeout policies, and record key session metadata (e.g., timestamp, IP address).
- Track consecutive failed login attempts and enforce automatic account lockout policies once a defined threshold is reached.
- Implement and enforce RBAC rules with hierarchical permissions, ensuring access policies align strictly with the user's role and access level.
- Log all authentication attempts, registration actions, user management changes, authorisation decisions, and session activities to the Auditing Service for compliance and review.

Event Bus

The Event Bus is a core infrastructure component that provides asynchronous, publish-subscribe messaging capabilities across the entire disaster response system. It enables loosely coupled communication between services by allowing publishers to emit events without needing to know which subscribers will consume them. The Event Bus ensures reliable message delivery, supports event filtering and routing, and maintains event ordering when required. It acts as the nervous system of the application, facilitating real-time coordination between components such as Dispatching, Traffic Management, Evacuation Planning, and Notification during disaster scenarios.

Responsibilities

- Provide publish-subscribe messaging infrastructure for inter-service communication
- Accept event publications from any authorised service component
- Route events to all relevant subscribers based on topic subscriptions and event filters
- Guarantee reliable event delivery with configurable delivery semantics (at-least-once, exactly-once)
- Maintain event ordering within topics when required by business logic
- Support event filtering and content-based routing to reduce unnecessary message processing
- Store events temporarily in durable queues to handle subscriber unavailability
- Monitor event flow metrics, including throughput, latency, and error rates
- Provide event audit trails for compliance and troubleshooting
- Handle backpressure and flow control to prevent subscriber overload
- Implement circuit breakers and retry policies for resilient message delivery

Scheduler

The Scheduler component manages the delayed or timed execution of background jobs. The Scheduler's primary function is to accept a job with a specified execution time and ensure it is reintroduced into the system for processing precisely when that time arrives. It acts as the timekeeper for deferred tasks.

Responsibilities

- Accepts requests for delayed execution and persists them in the task queue using the broker.
- Accurately holds the task until the designated scheduled time.
- At the scheduled time, it redirects the task payload and metadata to the appropriate channel in the Event Bus for immediate consumption.

Notification

The Notification component acts as the central broadcast and alerting hub for the Response to City Disaster system, orchestrating multi-channel message delivery to all system actors during emergency operations. This component ensures that time-critical information reaches the appropriate recipients through reliable communication channels, supporting the operational continuity of disaster response efforts.

Responsibilities

- Orchestrates notification delivery via push (FCM/APNS), SMS (Twilio), email (SendGrid), and in-app WebSocket channels with intelligent fallback for failed deliveries.
- Manages four priority queues (P0-P3) aligned with disaster severity, ensuring sub-second delivery for critical P0 notifications with pre-emption capabilities
- Maintains versioned notification templates with variable substitution, channel-specific formatting, and multi-language support for rapid deployment

- Tracks end-to-end status, implements exponential backoff retry logic, and maintains audit-compliant delivery confirmation records
- Manages secure connections to external notification providers with circuit breaker patterns, adaptive throttling, and automated credential rotation
- Coordinates with the **Scheduler** component for deferred delivery, supports expiry windows, and enables batch processing for non-urgent communications

Data ETL

The Data ETL (Extract, Transform, Load) component is the primary data ingestion pipeline for the Response to City Disaster system. It is responsible for connecting to all heterogeneous external data sources, fetching raw data, and processing it into a clean, unified format. These sources include real-time feeds for traffic, public transport, emergency services, weather, utilities, and more, as consumed by the **Observability** component.

This component validates, transforms, and standardises all incoming data before loading it into the system's central data stores (e.g., time-series or geospatial databases). It works in conjunction with the Scheduler to run ingestion jobs at defined intervals and publishes events to the Event Bus upon data retrieval.

Responsibilities

- Manage configurations for all external data sources, including credentials, API endpoints, and data formats.
- Extract data by connecting to diverse external sources (e.g., REST APIs, streaming endpoints, file drops).
- Validate incoming raw data for schema compliance, completeness, and integrity.
- Transform and map data from various source-specific formats into a standardised, unified system schema.
- Load the clean, processed data into the appropriate persistence layers.
- Integrate with the **Scheduler** to execute periodic batch ingestion jobs (e.g., "poll traffic data every 60 seconds").
- Implement robust error handling, retry logic, and alerting for failed ingestion pipelines.
- Publish "data_ingested" events to the **Event Bus** to notify downstream services (e.g. to **Observability**) that new data is available.

Observability

The Observability component acts as a data supplier, serving ingested data from the Data ETL to a Dashboard and other consumer services. It provides the pulse of the disaster response effort by handling high-volume, real-time streaming data from diverse operational sources. It ensures that all services operate on reliable, accurate, and up-to-date information. This data is critical for providing an immediate operational picture during a disaster.

Responsibilities

- Provides real-time data for all kinds of external data sources.

- Supports paginated responses with robust filtering capabilities.
- Provides real-time query capabilities for Emergency Response Team (ERT) Members to inform immediate actions.
- Supplies critical disaster response data that can be consumed by other services to tackle the disaster.

Map

The Map Service provides the core geospatial APIs and data required for an interactive map interface. It aggregates real-time information on disasters, emergency services, evacuation routes, and traffic conditions, making this data available for the UI to display. It gathers data from both internal system components and external APIs, processes it, and provides endpoints for the UI to render it on the map. In addition, the service exposes processed geospatial data to other internal systems for further analysis and use.

Responsibilities:

- Provide geospatial APIs for rendering an interactive map interface.
- **Serve** real-time disaster incident locations.
- **Serve data** on the position and status of emergency units.
- **Provide data for visualising** evacuation routes and safe zones.
- **Provide data** on traffic detours and closed roads.
- Support geocoding (address and coordinates conversion).
- Manage and provide data for different information layers (disasters, traffic, evacuations, units).
- Provide real-time updates.
- Provide data endpoints that support client-side caching for offline mode.

Incident Registry

The Incident Registry acts as the system of record for all disaster-related incidents across the city. It captures, manages, and synchronises incident information in real time from **Observability**. It serves as the single source of truth for downstream components like Dispatching, Map, Notification, and Evacuation Planning, ensuring consistent situational awareness and coordinated response.

Responsibilities

- Create, store, and update disaster incident records with metadata such as type, severity, location, and timestamp.
- Maintain incident lifecycle states (e.g., *reported* → *verified* → *active* → *resolved* → *closed*).
- Integrate validation workflows for incident confirmation by authorised officials.
- Link related entities such as dispatched units, evacuation plans, and citizen reports.
- Stream updates in real time to subscribed services (e.g., from Map, Notification) via the Event Bus.
- Maintain geospatial indexing for efficient search and filtering by region or radius.

- Support prioritisation based on severity, population density, or infrastructure impact.
- Generate aggregated incident analytics and statistics for decision support.
- Log all operations to the **Auditing** function for traceability and compliance.

Communication Platform

The Communication Platform is the main system that allows everyone to stay connected during a city disaster. It helps emergency teams, control rooms, and citizens share information quickly, safely, and without delays. It works as a real-time connection layer, enabling crucial communication among the Response Teams themselves and facilitating interaction between users and the Support Bot Service.

This function makes sure that all messages and updates, such as alerts, reports, or team discussions, are delivered on time, recorded properly, and easy to track, so that everyone can work together smoothly during emergencies. It achieves this using different tools like APIs, WebSockets, and live data streams.

Responsibilities

- Enable two-way communication between responders, control rooms, and citizens.
- Facilitate seamless communication and coordination within the Response Team structure.
- Manage the communication channel between users and the dedicated Support Bot Service.
- Support chat, voice, and video communication for different emergencies.
- Allow real-time coordination between teams with instant updates.
- Send messages to the right team based on role, location, or incident type.
- Create and manage secure communication channels for each incident.
- Send alerts or announcements through the Notification component when needed.
- Save all communication history for reports and future review.
- Show who is online or available for better team coordination.
- Allow sharing of images, videos, and data related to incidents.
- Protect all messages with end-to-end encryption for security.
- Monitor system performance with communication analytics and reports.

Dispatching

The Dispatching component is responsible for the real-time assignment, coordination, and management of emergency response units to disaster locations. It ensures optimal allocation of available resources (ambulance, police, fire, and traffic management units) based on live data integration from city traffic conditions, emergency services availability, and incident priorities. The component continuously monitors unit status, with field units to minimise response time and maximise operational efficiency during disaster scenarios.

Responsibilities

- Receive and process emergency dispatch requests from ERT members.
- Maintain a real-time registry of all available emergency units and their current status (available, dispatched, en route, on-scene, offline).
- Assign appropriate emergency units to disaster incidents based on type, severity, proximity, and resource availability.
- Track unit locations and status throughout the dispatch lifecycle.
- Send dispatch notifications and instructions to assigned emergency units.
- Update dispatch status across all connected systems (map, dashboards, incident registry).
- Handle unit status updates from field personnel.
- Support simultaneous dispatch operations for multiple concurrent incidents.
- Maintain dispatch history for all operations.
- Coordinate with the **Routing** component for dynamic route recalculation when conditions change.
- Manage priority queuing when demand exceeds available resources.
- Provide dispatch analytics and performance metrics.
- Support manual override and reassignment capabilities for ERT members.

Traffic Management and Routing

The Traffic Management and Routing component acts as the authority for all movement during a disaster. It is responsible for both monitoring the real-time state of the road network and calculating optimal routes based on that data.

The service consolidates with the **Observability** component to gather traffic sensor data, congestion feeds, incident alerts, and road closure inputs to dynamically update the transportation network. It then uses this live data to compute optimal, safe, and efficient paths for emergency response units and civilian guidance. It dynamically incorporates traffic disruptions, road closures, evacuation constraints, and hazard zones, adapting routes in real time as conditions change.

Responsibilities:

- Ingest real-time traffic flow, congestion alerts, and road status telemetry.
- Detect traffic blockages, bottlenecks, and hazard zones.
- Enforce road closures and vehicle movement restrictions during emergencies.
- Compute optimal routes for emergency units and evacuation traffic.
- Generate and publish detour directives, road closures, and route updates to the Map service and other components.
- Support continuous re-routing under changing disaster conditions.
- Prioritise emergency vehicles with dedicated routing logic.
- Maintain a real-time traffic map and historical congestion logs.
- Provide route safety checks, validation, and route caching.
- Support hazard-aware evacuation paths.
- Support manual overrides by emergency authorities.
- Notify the public and emergency responders about closures and detours.
- Integrate with external traffic data providers when available.

Evacuation Planning

The Evacuation Planning component is responsible for generating, updating, and managing evacuation plans in real time during emergencies. It consolidates information from **Observability** and **Incident Registry**, such as incident data, real-time traffic conditions, available resources, and historical records, to produce optimised and actionable evacuation plans. These plans define safe routes, resource allocations, and communication strategies to guide both emergency responders and civilians.

Once validated by the Emergency Manager, the evacuation plan is distributed to relevant services, including components such as **Mapping, Dispatching, and Notification**, ensuring coherent, coordinated execution across the city's emergency response network.

Responsibilities:

- Generate optimized, multi-part evacuation plans by modeling real-time data on incidents, traffic, available resources, and environmental conditions.
- Continuously assess and adapt active plans as new, dynamic data becomes available from other services.
- Identify and allocate resources efficiently, such as shelters, transport units, and personnel, as a core part of the plan.
- Enforce a human validation workflow, presenting the optimized plan to an Emergency Manager for final approval before execution.
- Coordinate the system-wide publication of the approved plan to the **Map, Notification, and Dispatching** components.
- Store and reuse knowledge from past events, simulations, and plan outcomes to improve the effectiveness of future recommendations.
- Ensure all generated plans, approvals, and publications are logged for traceability.

Support System

The Support System provides psychological assistance, crisis guidance, and verified disaster-related information to citizens during emergencies. It offers an automated conversational assistant capable of delivering emotional support, answering crisis queries, reducing panic, and directing individuals to official emergency services when required. The system ensures sensitive and safe response behaviour, prioritises user wellbeing, and escalates cases to human responders when distress indicators are detected. It integrates real-time data from internal services to provide accurate situational updates and safety recommendations.

Responsibilities:

- Provide emotional reassurance and crisis communication to affected individuals.
- Deliver verified disaster information and guidance from an official source.
- Detect distress, self-harm intent, or panic signals and trigger escalation workflows.
- Route critical emergencies to human helplines or medical responders.
- Offer multi-lingual and accessible communication support.

- Maintain conversation logs for ethical compliance and trend monitoring.
- Integrate real-time incident and evacuation updates into chatbot responses.
- Prevent misinformation and enforce safe content policies
- Maintain privacy and ensure secure handling of conversations

Auditing

The Auditing component acts as a logger, recording all critical audit events from every microservice and operational system as they occur. The data collected must be Immutable to ensure a non-repudiable log of events for post-incident analysis and compliance. Logging should be implemented asynchronously to prevent audit logging from adding unnecessary latency to core operational services, with events being streamed reliably to a dedicated database.

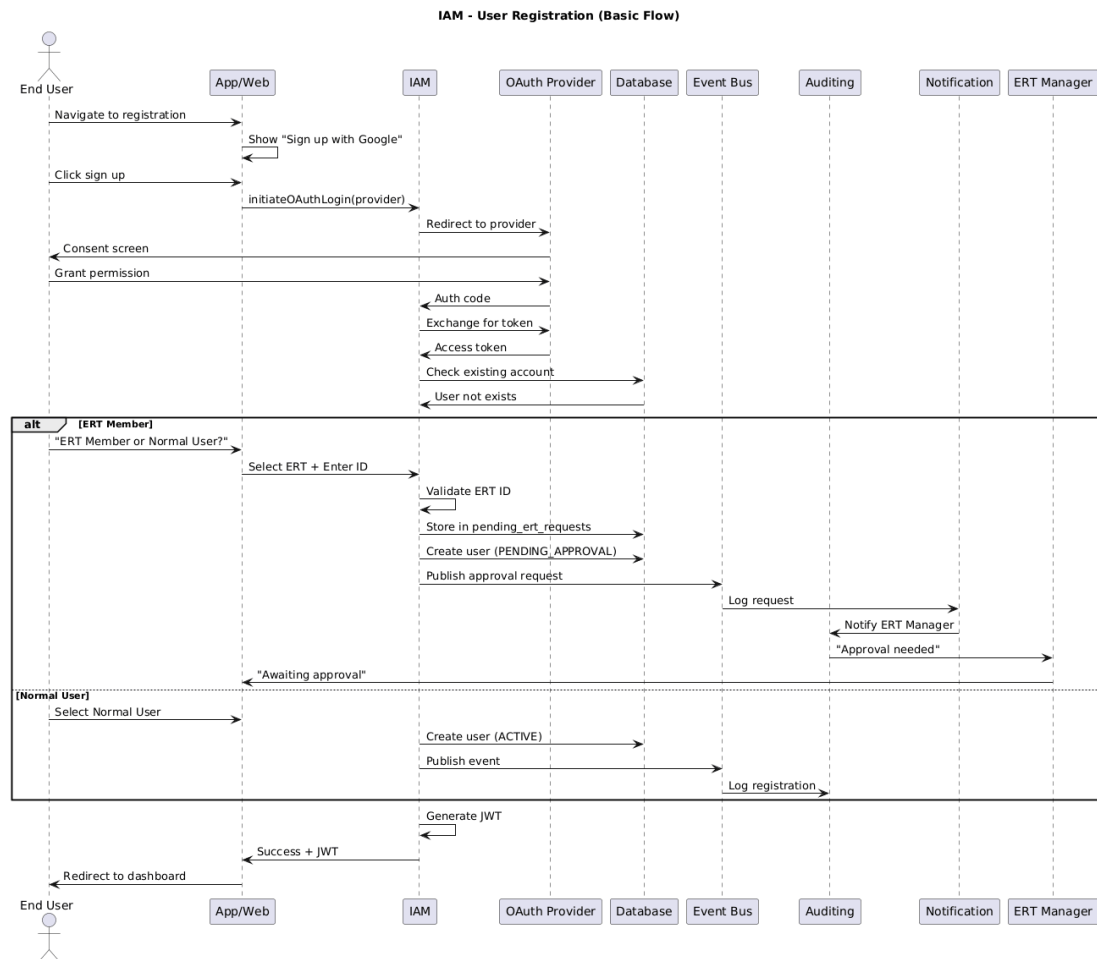
Responsibilities

- Receiving and timestamping audit events from all internal and external services.
- Persisting audit logs in an immutable, append-only data store.
- Providing secure, indexed query access for historical analysis and compliance.

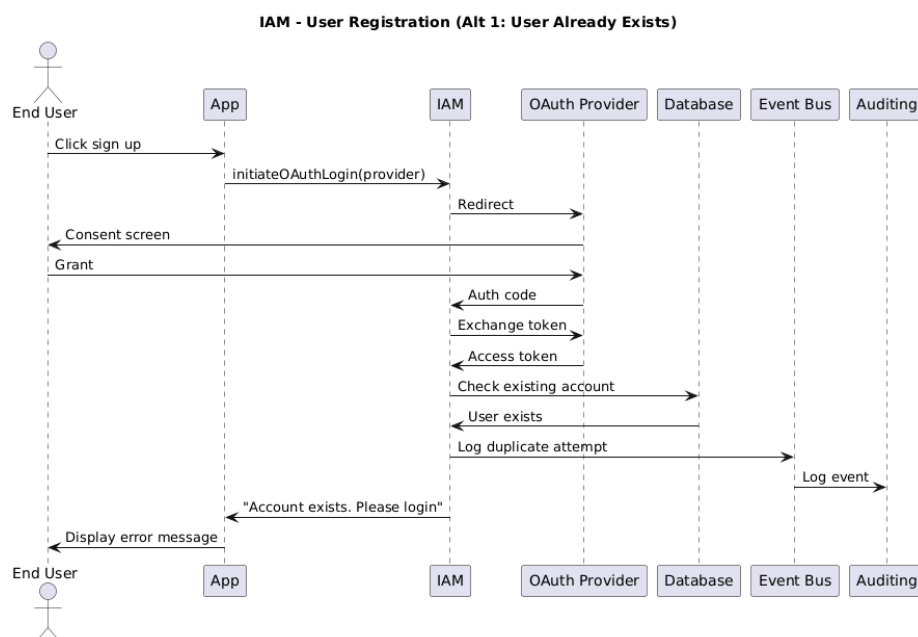
1.3. Component interactions

Identity Access Management

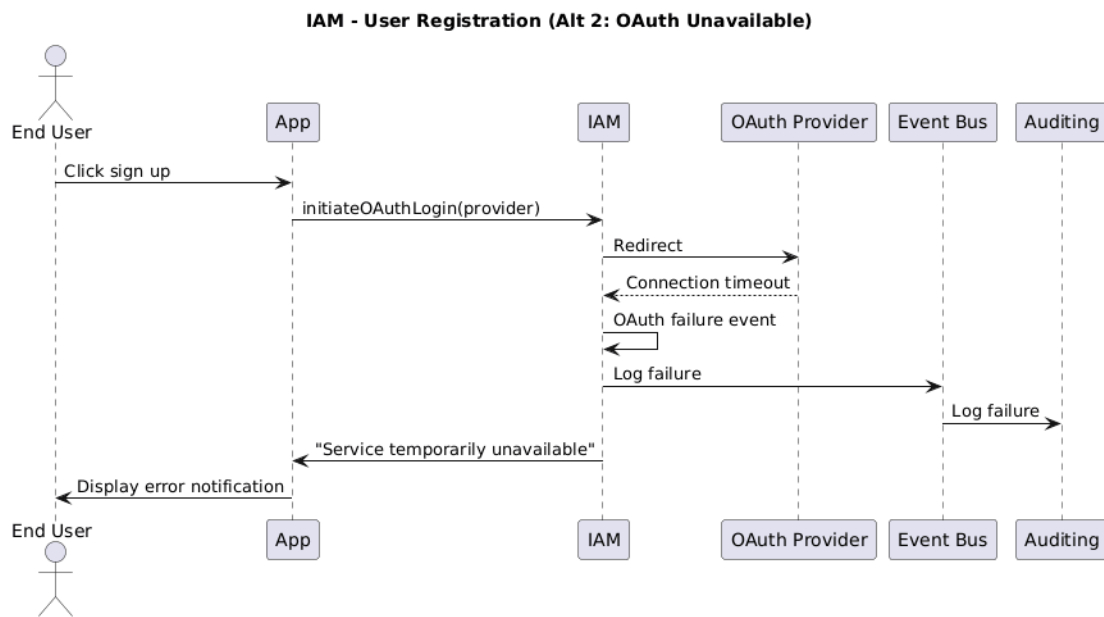
Basic Flow 1: User Login



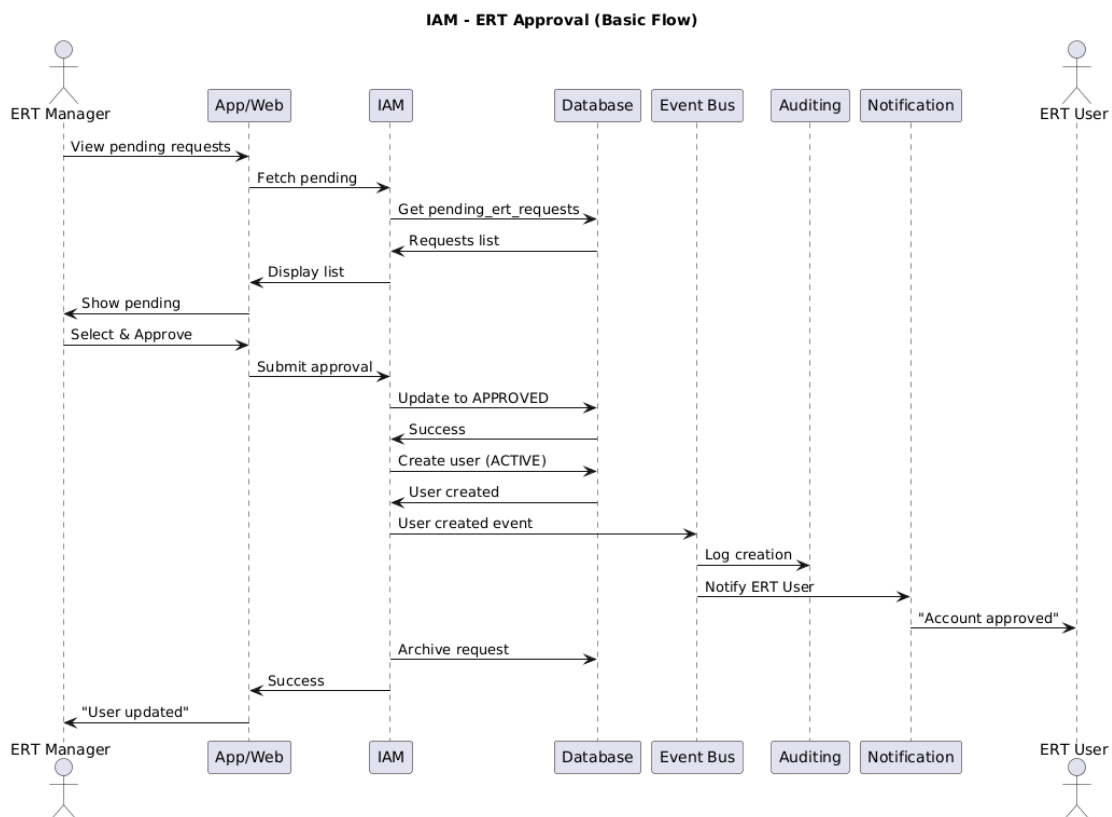
Alternate Flow:



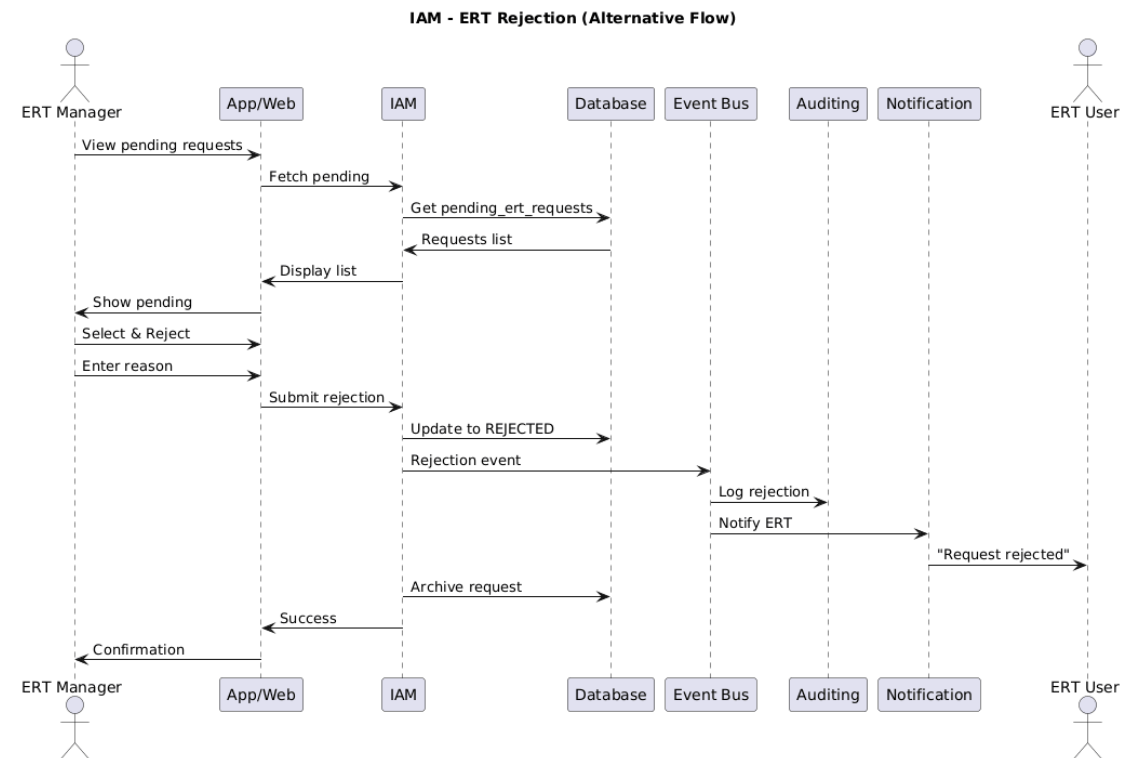
Alternate Flow 2:



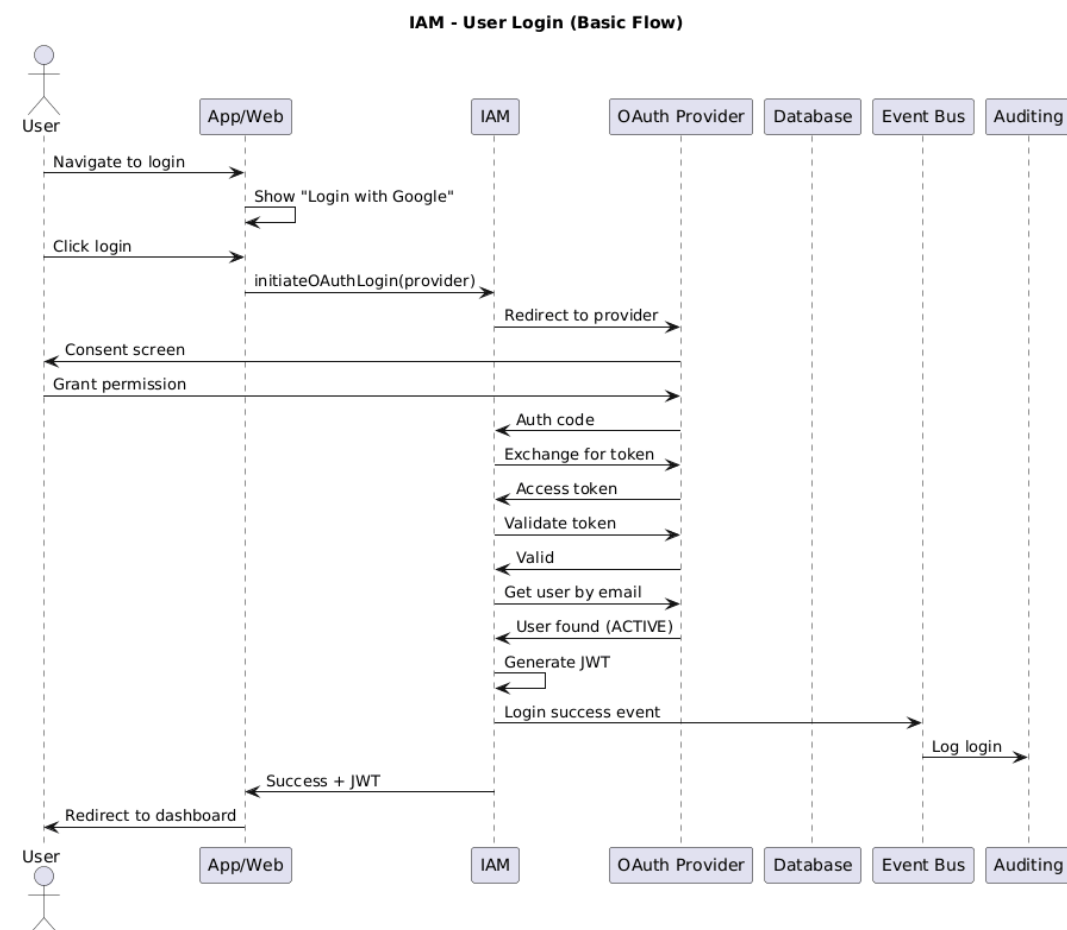
Basic Flow : ERT Approval



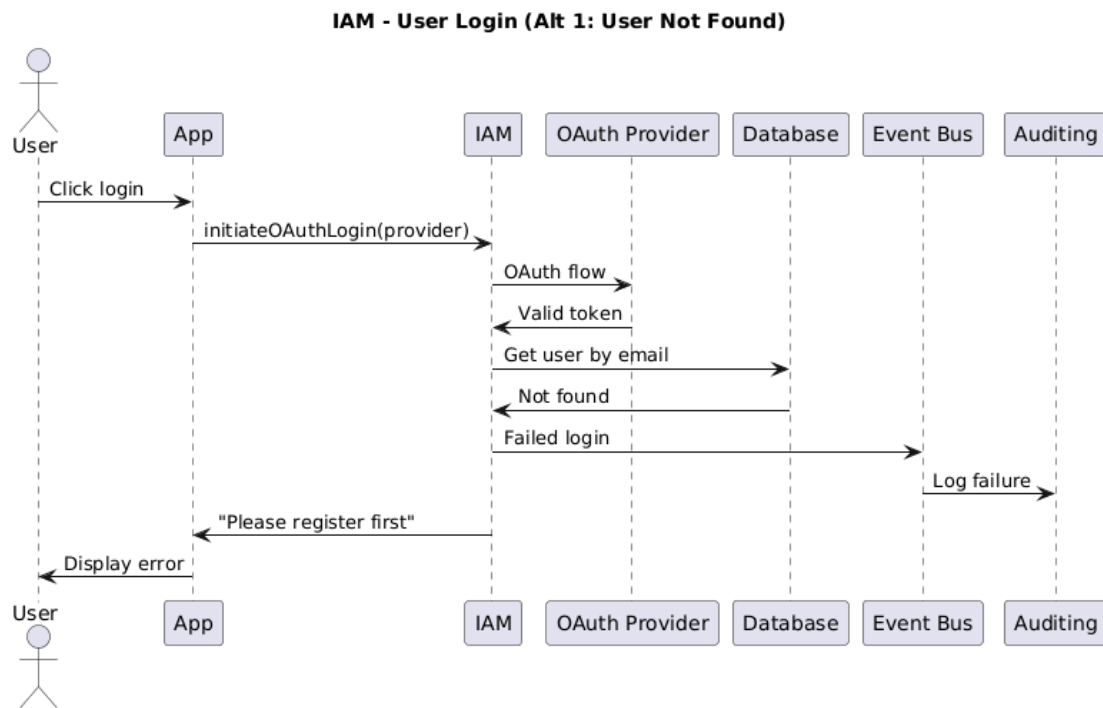
Alternate Flow 1:



Basic Flow 3: User Login

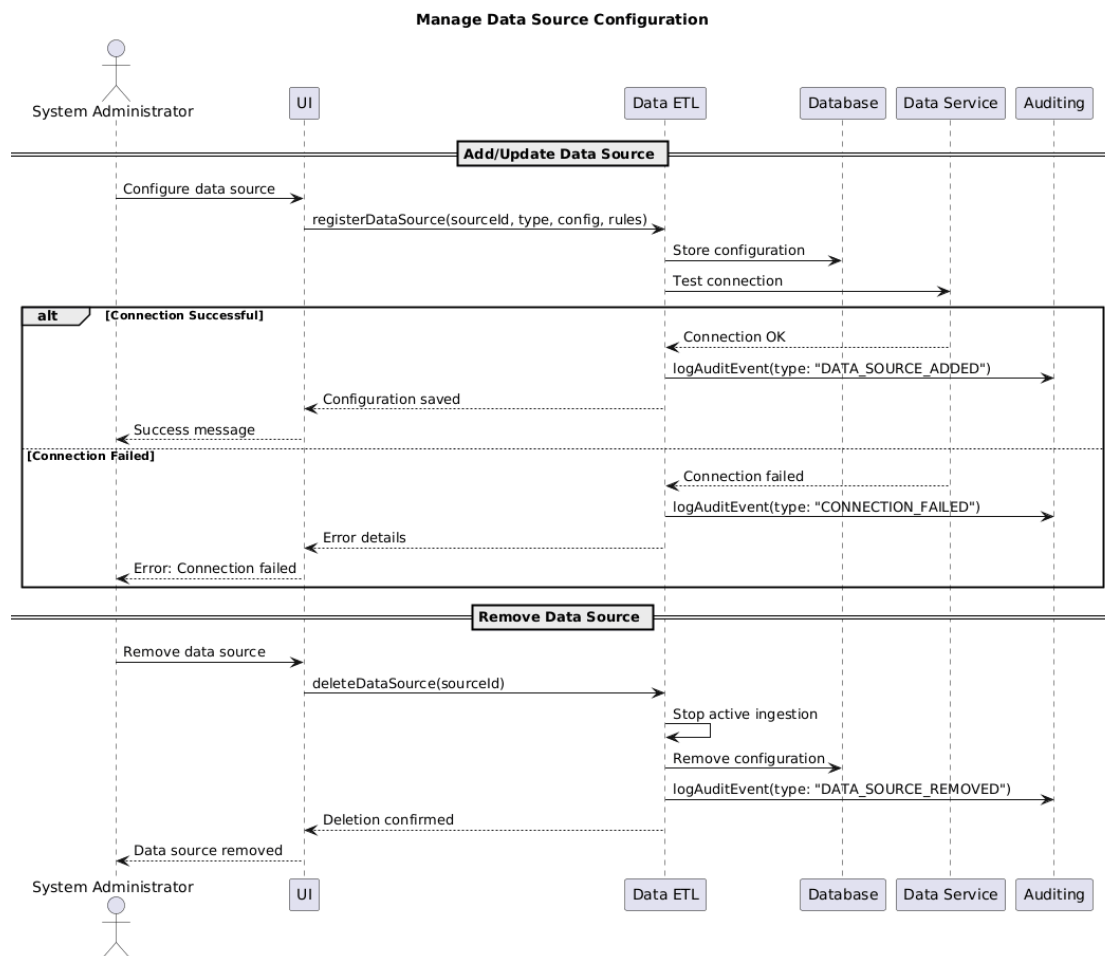


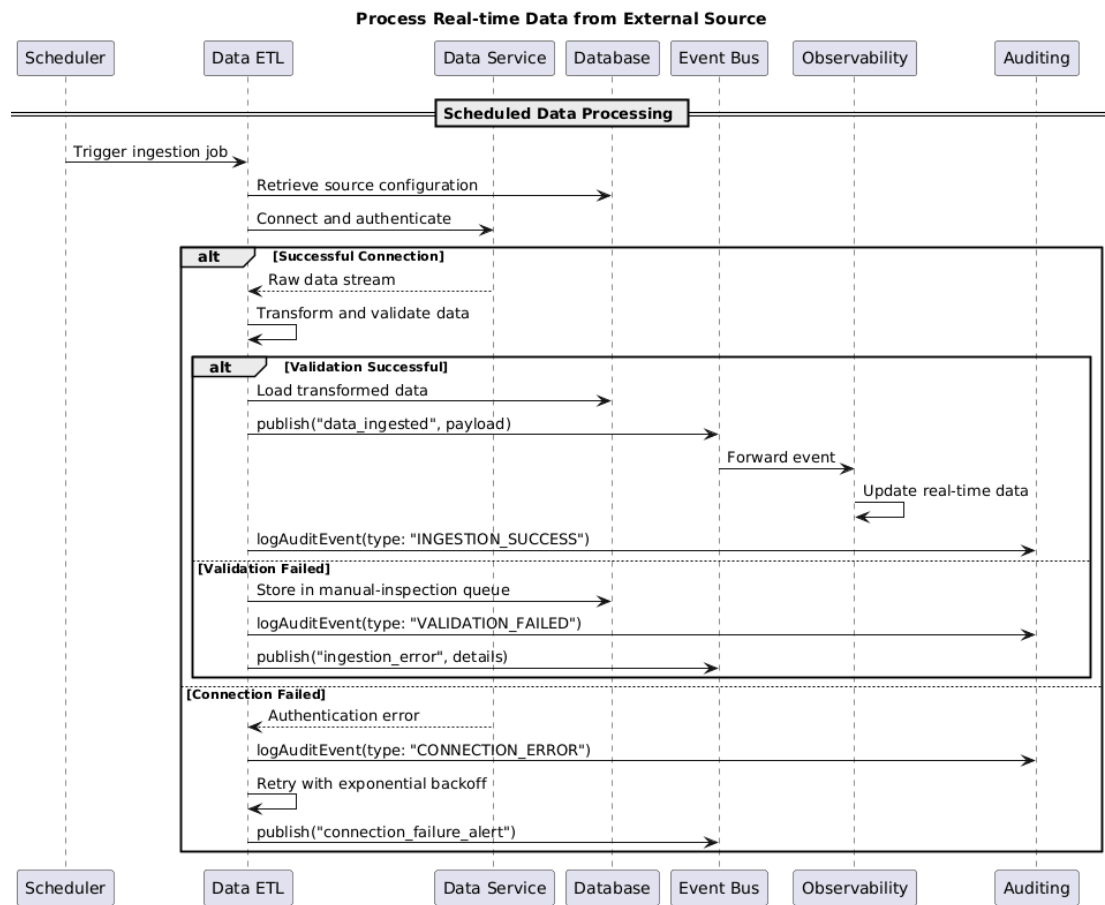
Alternate Flow:



Manage and Process Real-time Data Sources

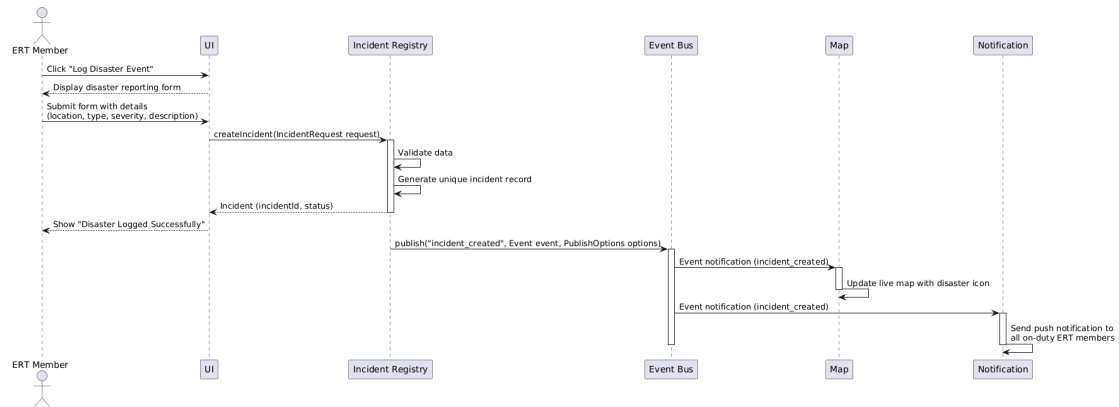
Basic Flow: Manage Data Sources



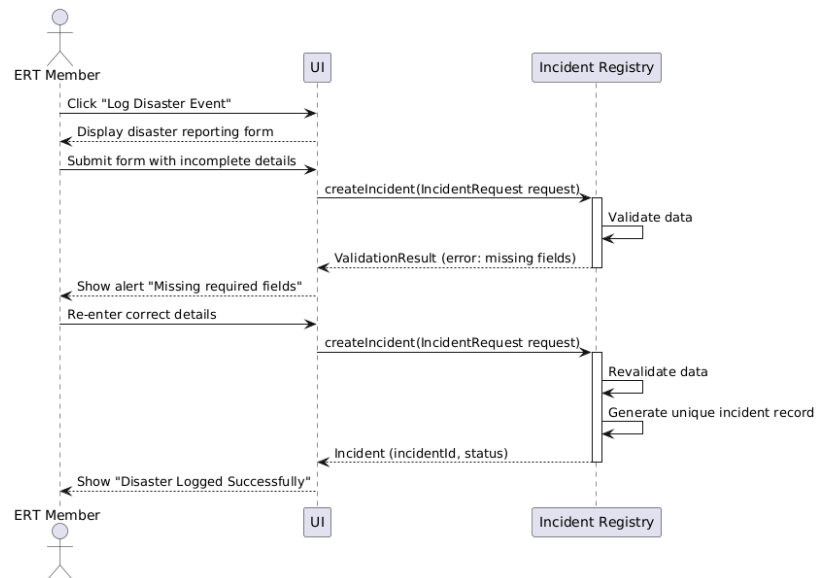
Basic Flow 2: Process Data from Source

Log Disaster Event

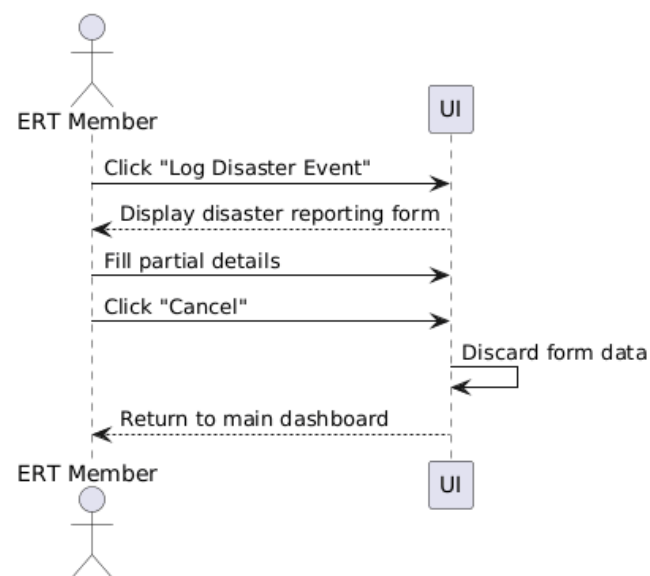
Basic Flow:



Alternate Flows :

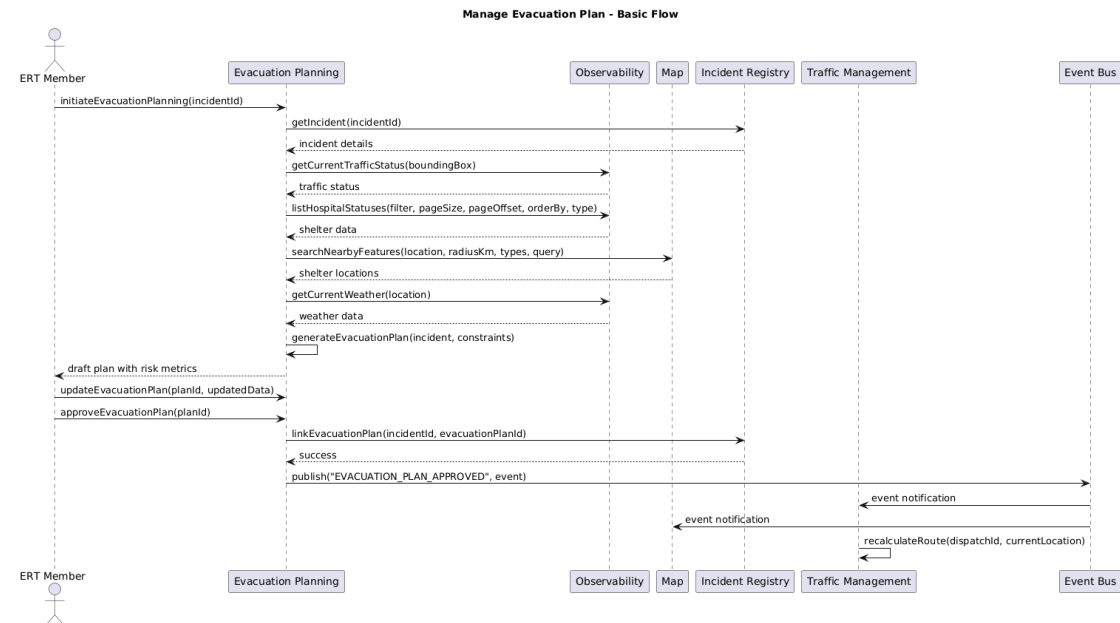


Alternate Flow 2:

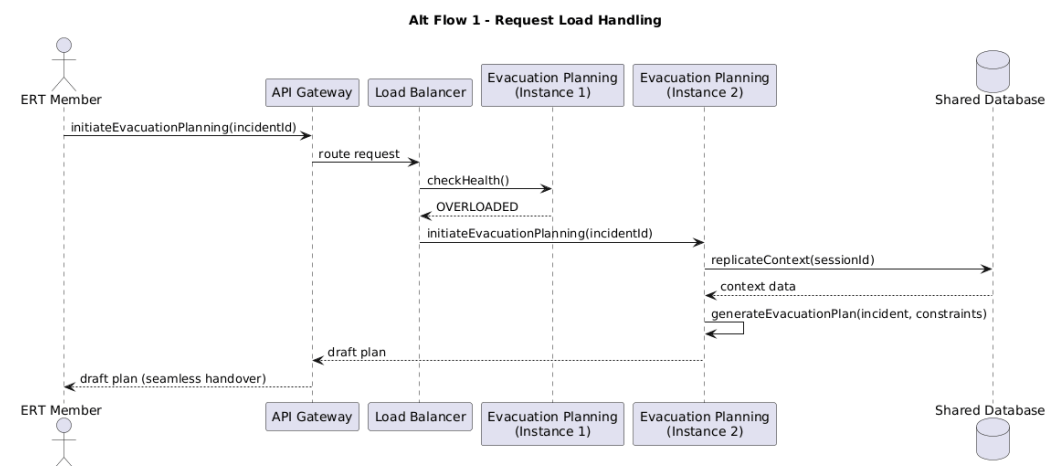


Manage Evacuation Plan

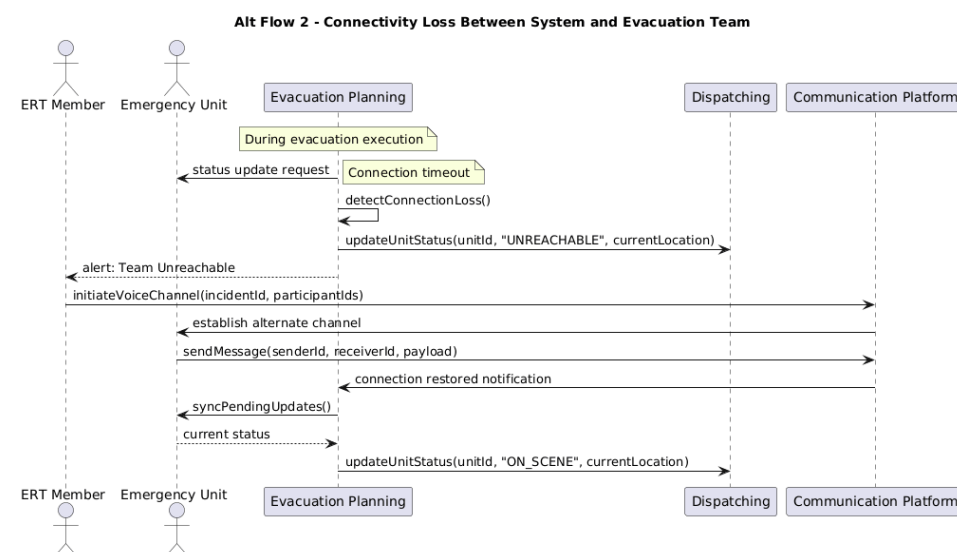
Basic Flow:



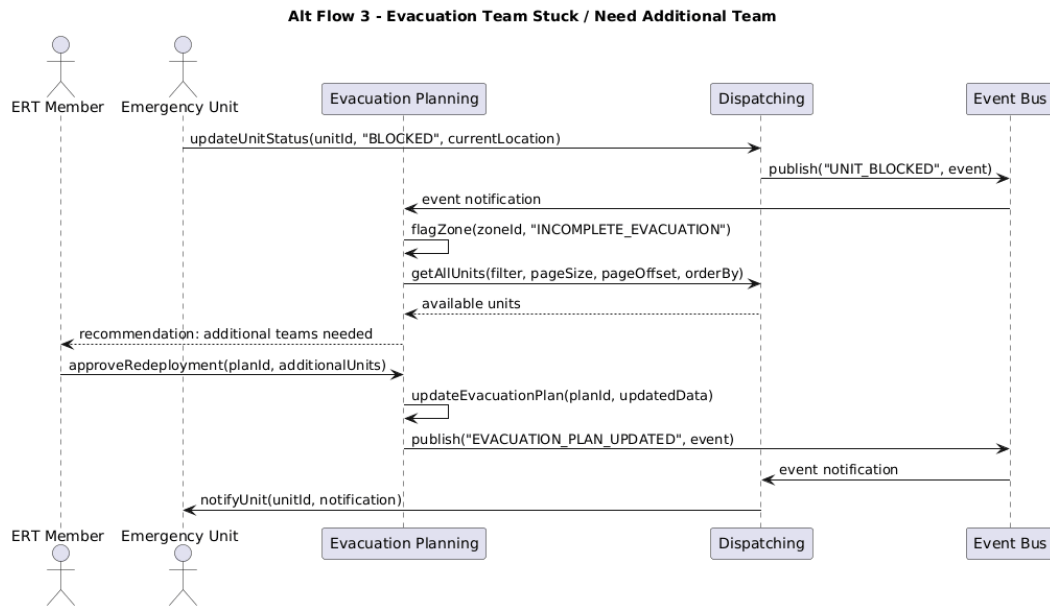
Alternate Flow 1:



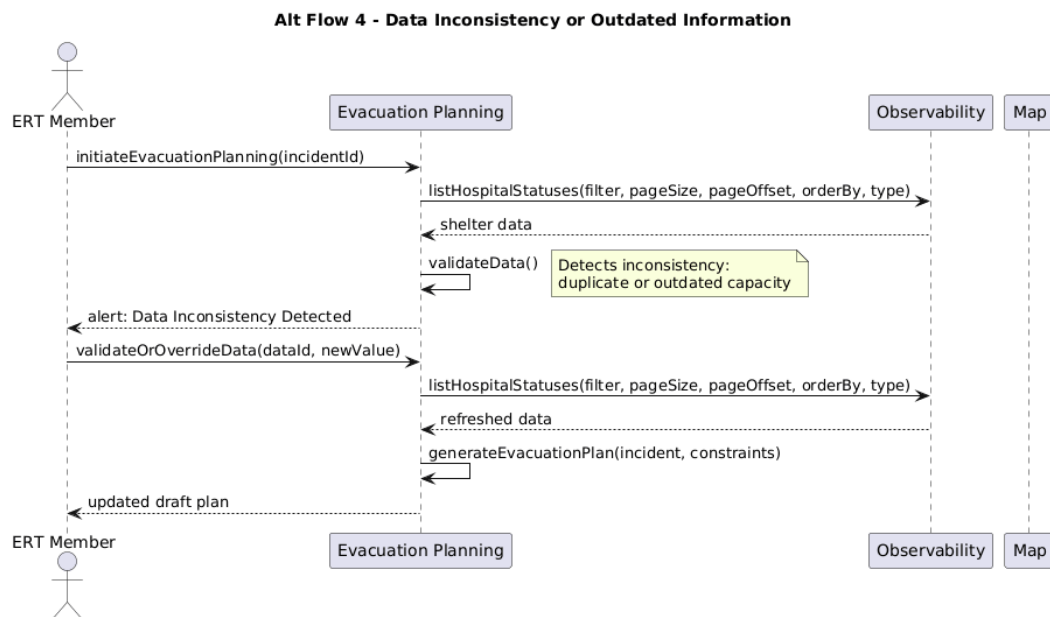
Alternate Flow 2:

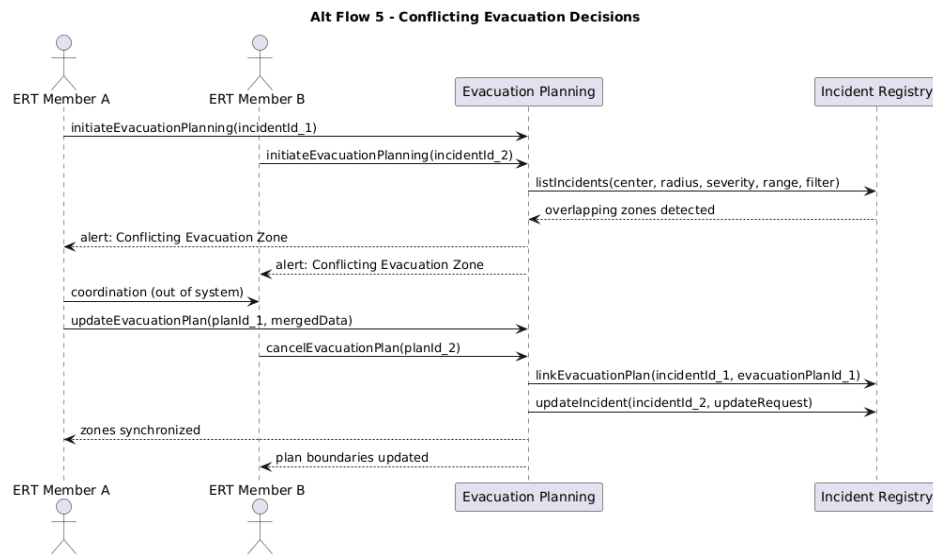


Alternate Flow 3:



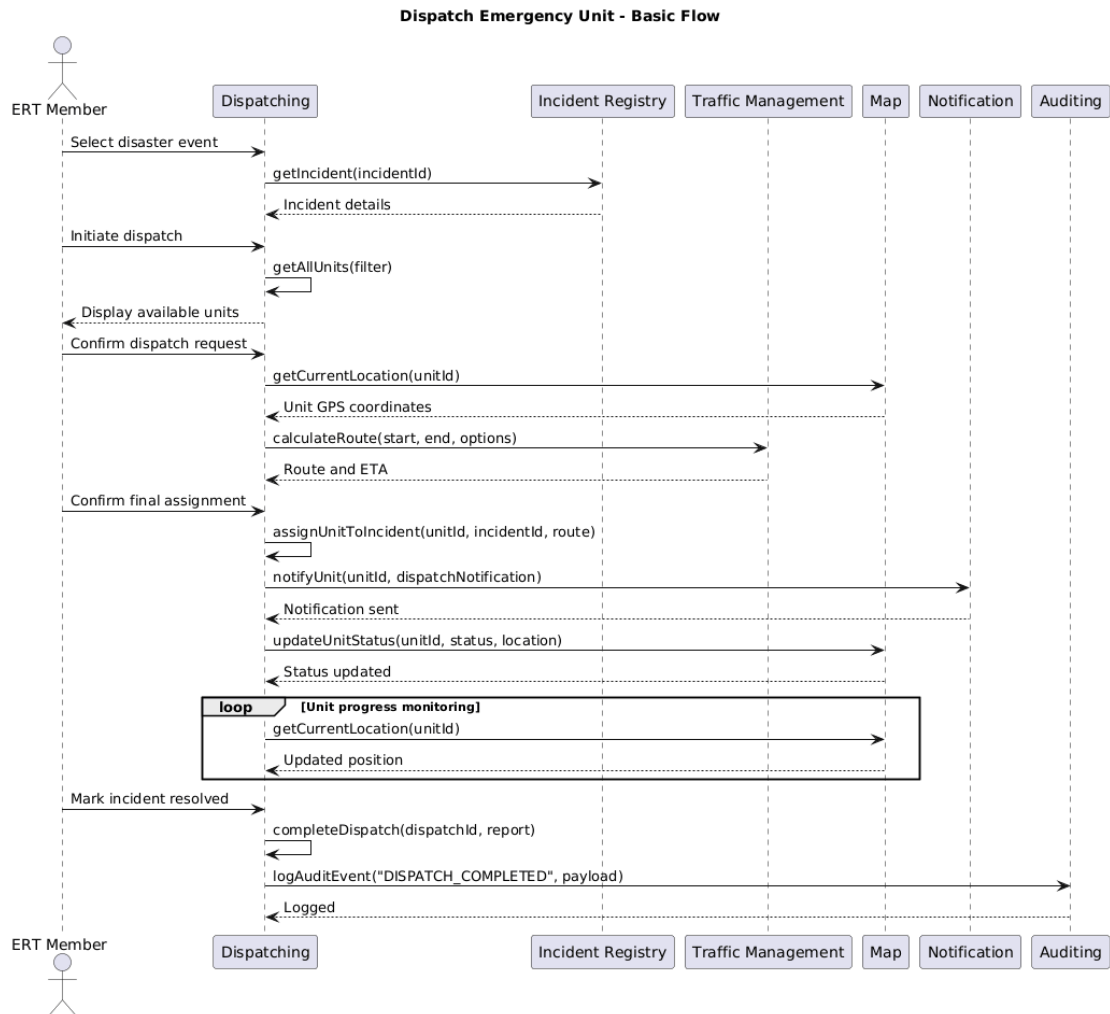
Alternate Flow 4:



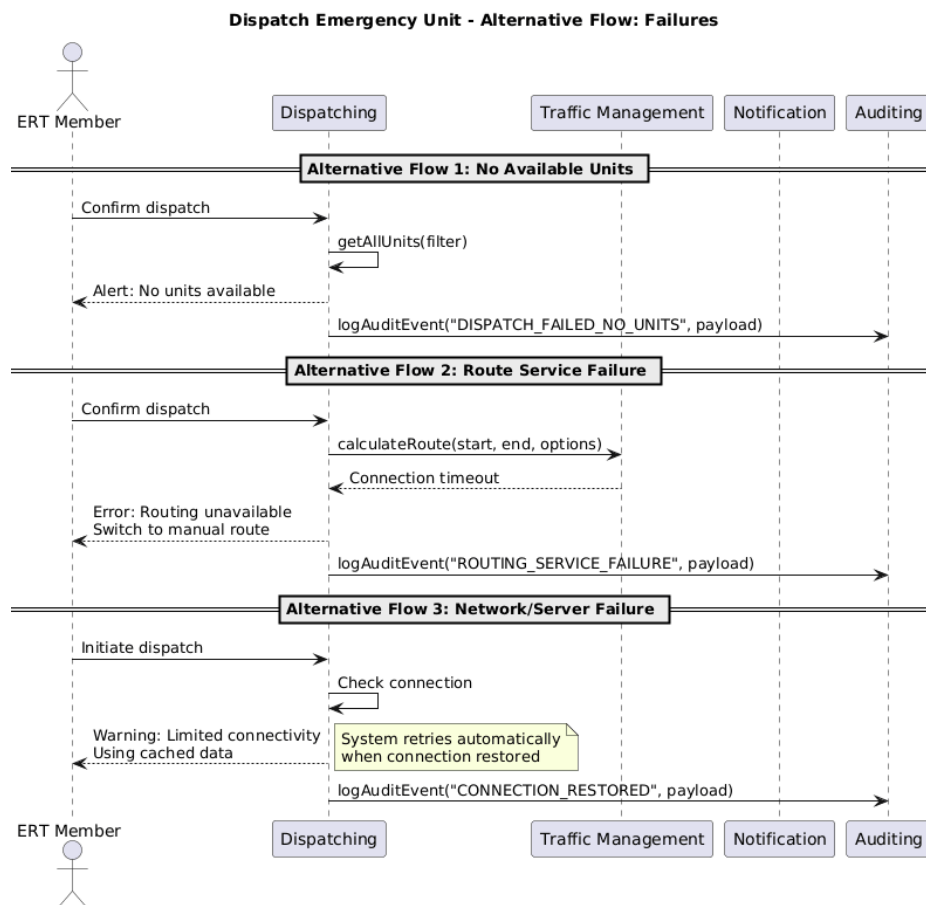
Alternate Flow 5:

Dispatch Emergency Unit

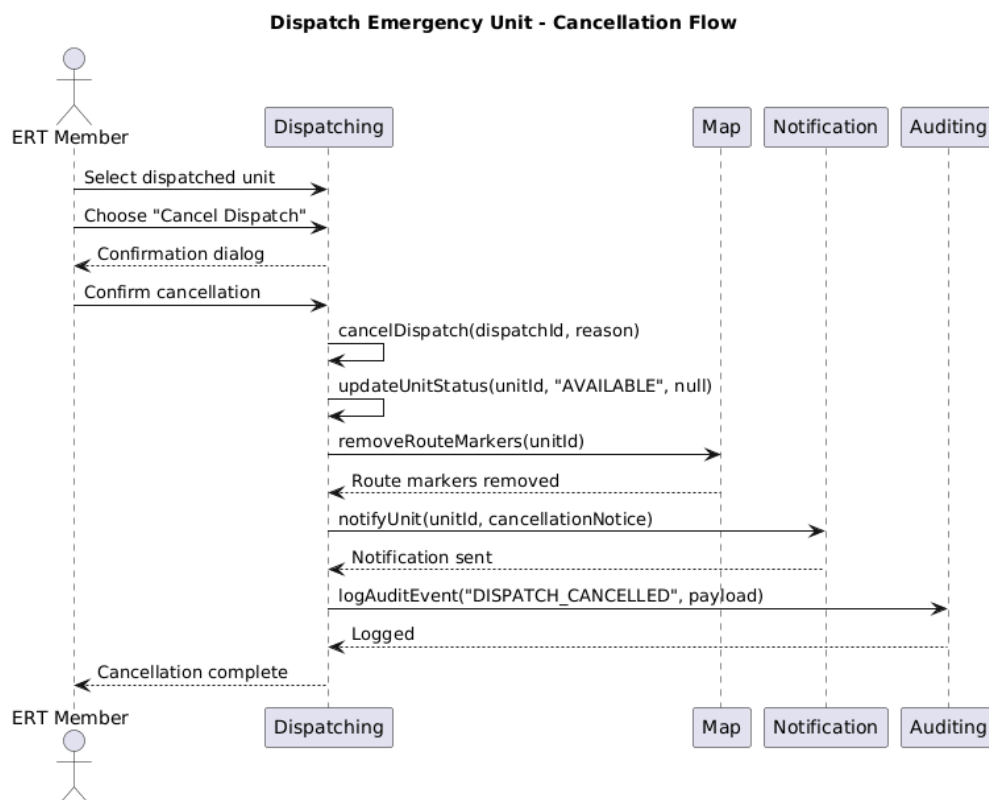
Basic Flow:



Alternate flow: Different Failures

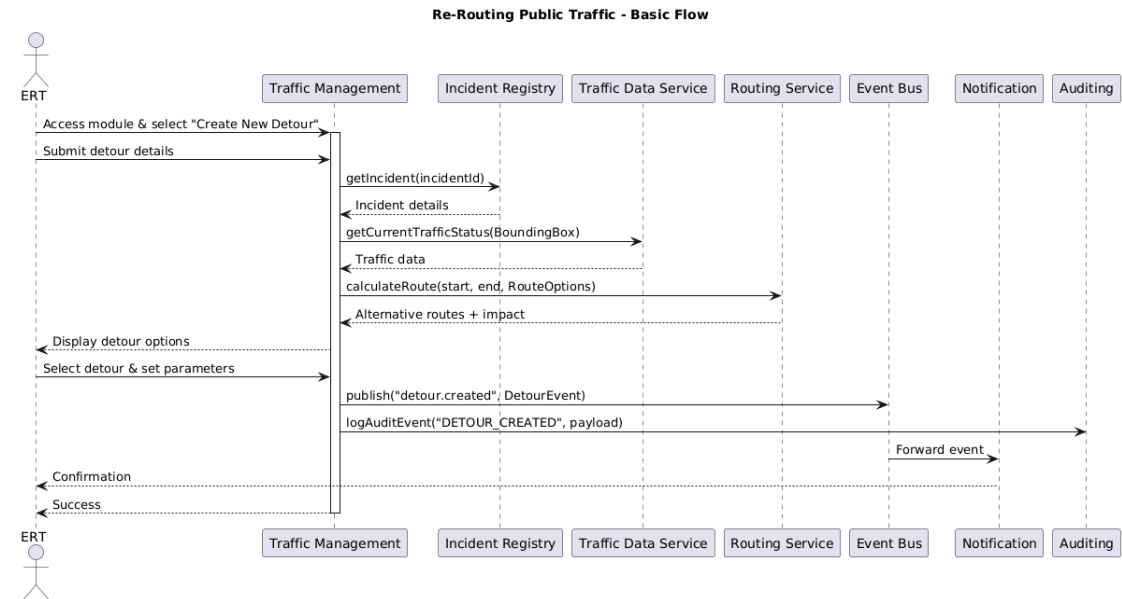


Alternate flow : Cancellation of Dispatch

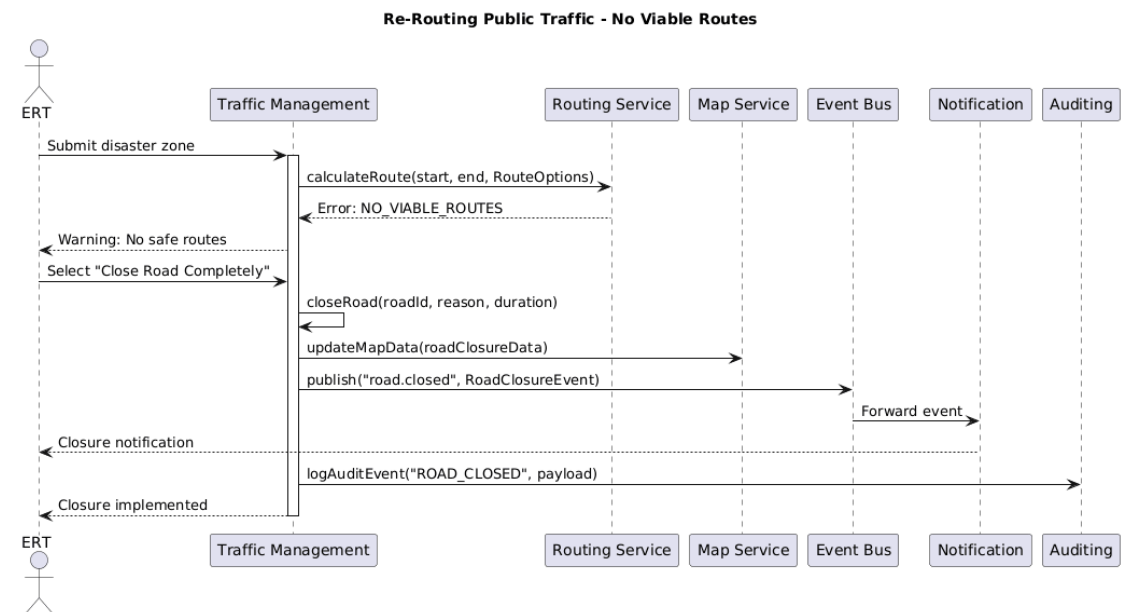


Re-Routing Public Traffic

Basic flow

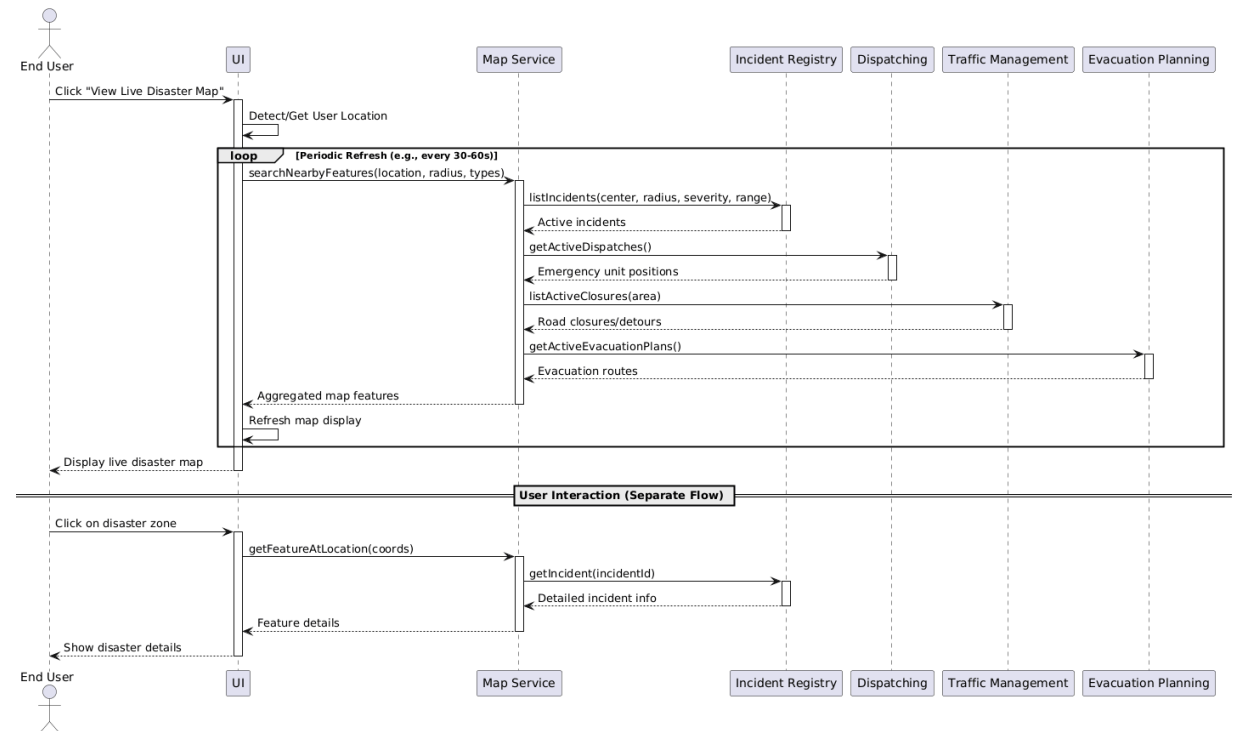


Alternate Flow : No viable route

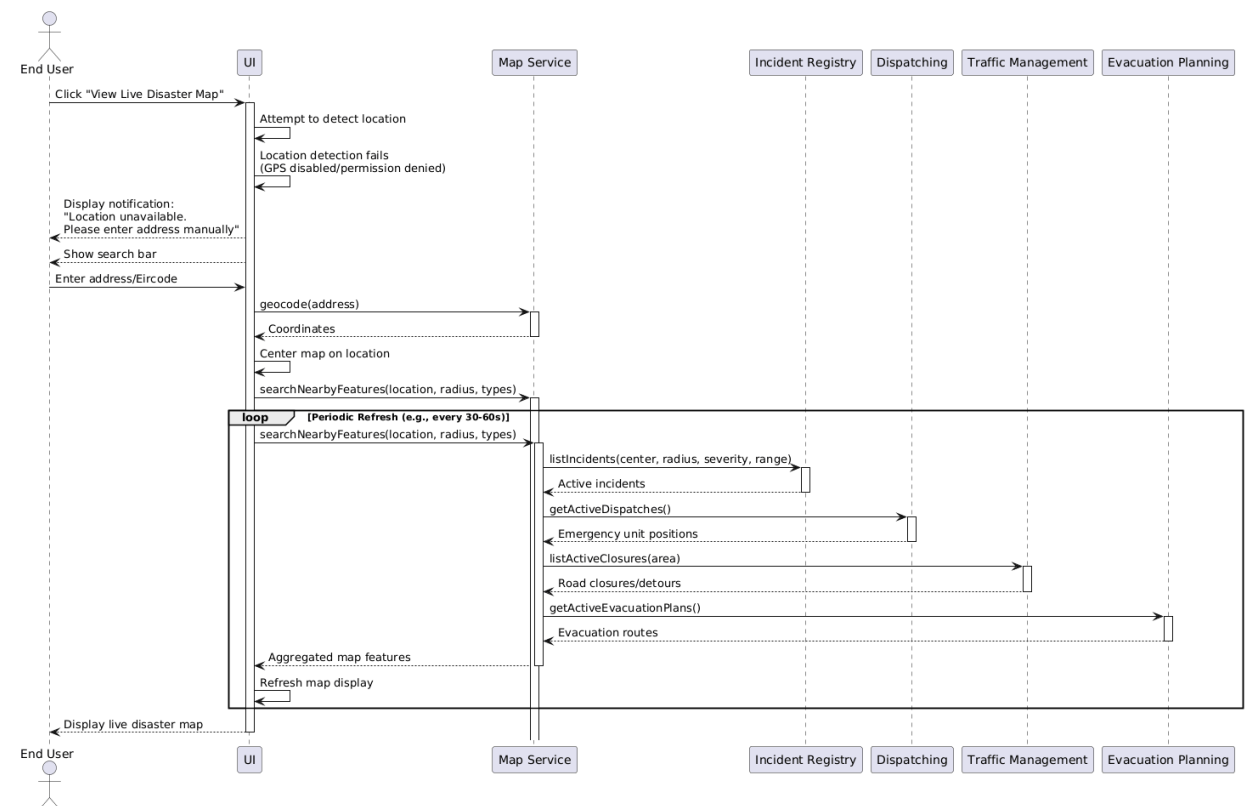


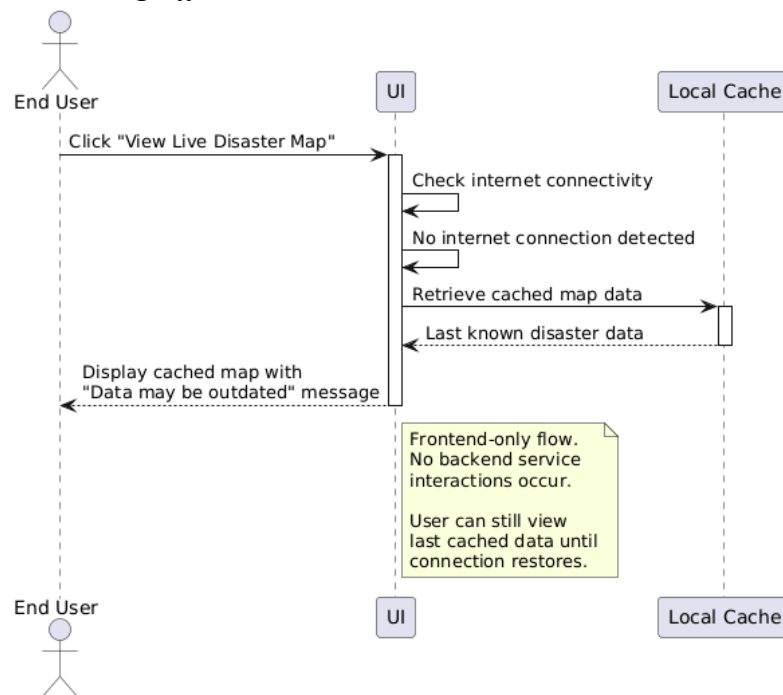
View Live Disaster Map

Basic Flow: Flow to see live disaster map



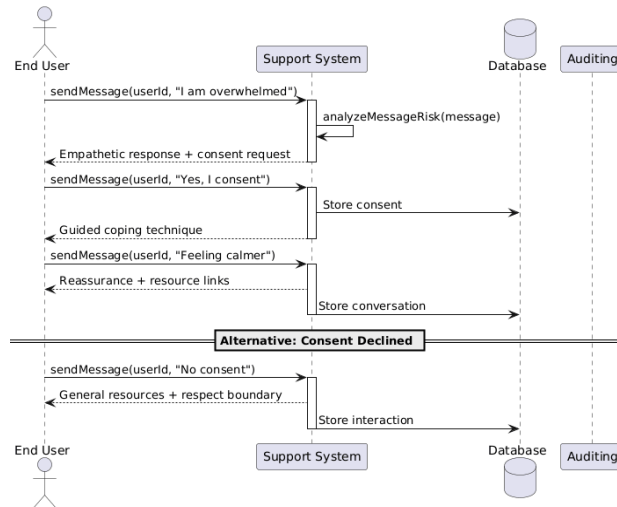
Alternate Flow: When User Denies Location



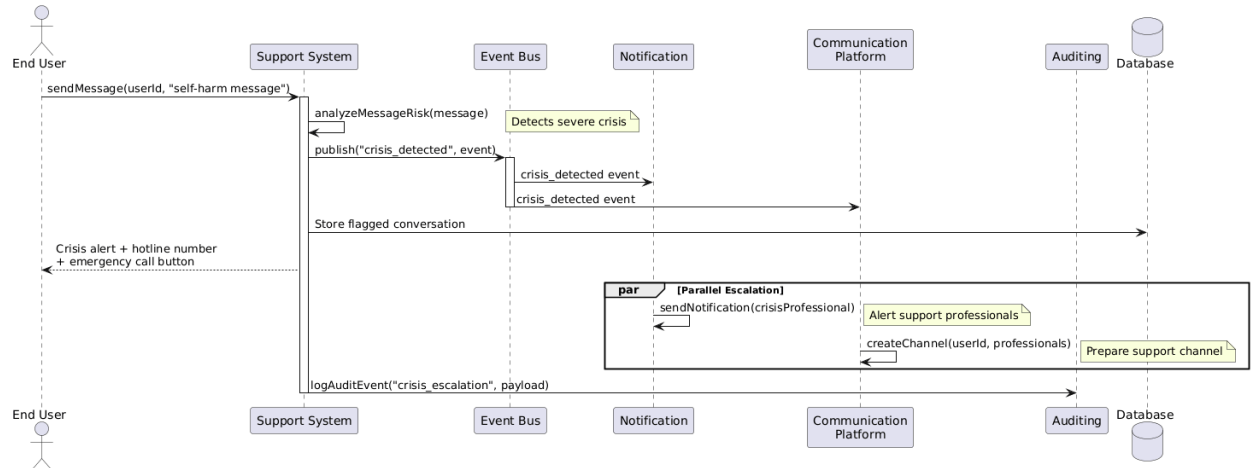
Alternate Flow: During Offline

Crisis Emotional Support and Information Bot

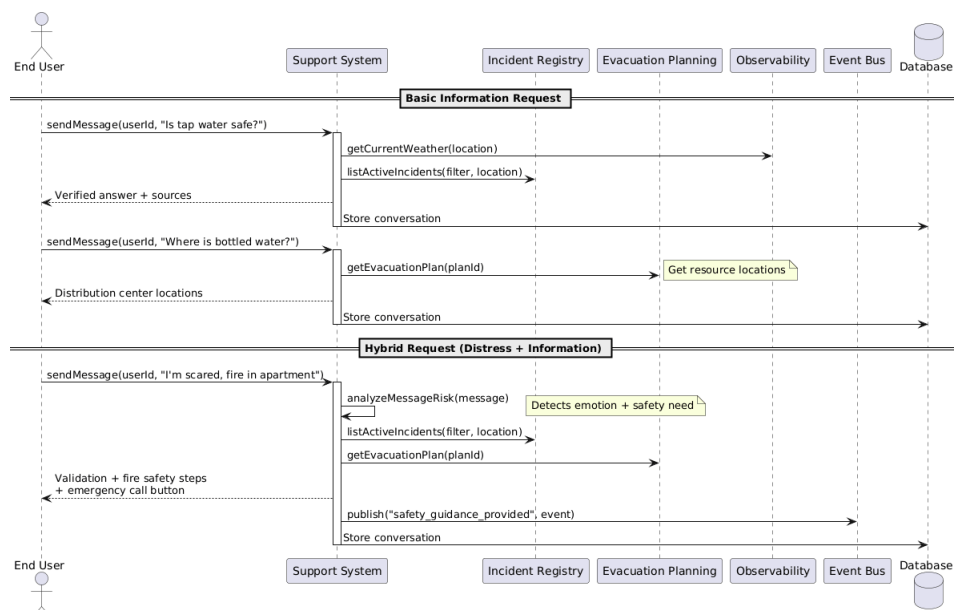
Basic Flow 1: End User's interaction with the Support Bot



Basic Flow 2: Severe Crisis Detection



Basic Flow 3: End User's hybrid interaction with the Support Bot



1.4. Component APIs

Identity and Access Management (IAM)

OAuth Authentication Flow APIs

- *OAuthRedirectResponse initiateOAuthLogin(String provider)*
Responsibilities: Initiates OAuth login by generating and redirecting to the provider's authorisation URL with required parameters.
- *AuthenticationResponse completeOAuthLogin(String provider, String authorizationCode)*
Responsibilities: Completes OAuth login flow-exchanges authorisation code for tokens, retrieves/creates user account, generates application JWT, and establishes session.
- *void createUpdateUserProfile(CreateUpdateProfileRequest profile)*
Responsibilities: After OAuthLogin creates a user profile, this API can be used to update the user profile.
- *void addERTDetails(ERTDetailsRequest ertDetails)*
Responsibilities: For users registering as ERT members, adds ERT credentials (ERT ID), validates against the registry, and updates account status to PENDING_APPROVAL.
- *AuthenticationResponse refreshSession(String refreshToken)*
Responsibilities: Refreshes expired sessions and issues a new application JWT.

Authorisation Flow APIs

- *AuthorizationResult authorizeRequest(String token, String requestedAction)*
Responsibilities: Validates JWT token, verifies active session, extracts user permissions (role and access level), and determines if action is permitted.

Account Management Flow APIs

- *void manageERTApproval(String userId, ApprovalDecision decision)*
Responsibilities: Handles ERT member approval/rejection workflow, including status updates and notifications.
- *void manageAccountStatus(String userId, AccountAction action)*
Responsibilities: Handles account status changes (lock, unlock, suspend) and associated cleanup operations.

Event Bus

- *void publish(String topic, Event event, PublishOptions options)*
Responsibilities: Publishes an event with specific options such as priority level, time-to-live (TTL), delivery guarantees (at-least-once/exactly-once), and custom headers.
- *String subscribe(String topic, EventFilter filter, EventHandler handler)*

Responsibilities: Registers a filtered subscription where only events matching the filter criteria are delivered to the handler. Supports filtering by event type, priority, source, or custom attributes.

- *void unsubscribe(String subscriptionId)*

Responsibilities: Removes an active subscription, stopping event delivery to the associated handler.

- *void createTopic(String topic, TopicConfig config)*

Responsibilities: Creates a new event topic with specified configuration, including retention policy, partition count, replication factor, and access controls.

- *void deleteTopic(String topic)*

Responsibilities: Remove a topic and all its associated events and subscriptions. Requires administrative privileges.

- *void publishBatch(String topic, List<Event> events)*

Responsibilities: Publishes multiple events as a single atomic operation for improved performance and consistency when publishing related events.

Scheduler

- *void schedule(Data payload, String scheduledTime, String targetTopic)*

Responsibilities: Submits a new task for delayed execution, ensuring the 'payload' is delivered to the 'targetTopic' via the Event Bus at the 'scheduledTime'.

Notification

Core Notification APIs

- *NotificationResponse sendNotification(NotificationRequest request)*

Responsibilities: Sends a single notification to specified recipients through configured channels. Primary API for immediate notification delivery with support for multiple channels, priority levels, and acknowledgement requirements.

- *BroadcastResponse broadcastNotification(BroadcastRequest request)*

Responsibilities: Sends notifications to large groups based on geographic zones, roles, or disaster-affected areas. Optimised for high-volume, simultaneous delivery with batch processing capabilities.

Template Management APIs

- *Template createTemplate(TemplateRequest request)*

Responsibilities: Creates a new notification template for reusable messaging patterns. Supports variable placeholders, multi-language content, and category-based organisation for disaster scenarios.

- *List<Template> listTemplates(String category, String language)*

Responsibilities: Retrieves available notification templates, optionally filtered by category (e.g., DISASTER_ALERT, EVACUATION) and language for template selection and management.

- *Template updateTemplate(String templateId, TemplateRequest request)*

Responsibilities: Updates an existing notification template. Creates a new version while maintaining template history for audit purposes.

- *void deleteTemplate(String templateId)*

Responsibilities: Marks a template as inactive. Templates are soft-deleted to maintain referential integrity with historical notifications.

Acknowledgement and Feedback APIs

- *AckResponse acknowledgeNotification(String notificationId, String userId, AckData data)*
Responsibilities: Records user acknowledgement of notification receipt with optional response data and location. Critical for tracking emergency responder confirmations and dispatching decisions.
- *List<Acknowledgment> getAcknowledgments(String notificationId)*
Responsibilities: Retrieves all acknowledgements for a specific notification, essential for monitoring emergency responder status and compliance tracking.

Notification History APIs

- *List<Notification> getNotificationHistory(String userId, TimeRange range, Integer pageSize, Integer pageOffset)*
Responsibilities: Retrieves paginated notification history for a specific user, useful for the user-facing notification inbox and historical tracking.

Batch Operations APIs

- *BatchJobResponse submitBatchJob(BatchJobRequest request)*
Responsibilities: Submits a large-scale notification batch job for asynchronous processing, returning job ID for status tracking. Used for scheduled notifications or large broadcasts.
- *BatchJobStatus getBatchJobStatus(String jobId)*
Responsibilities: Retrieves current status and progress metrics for a batch notification job, including processed count, success rate, and estimated completion time.
- *void cancelBatchJob(String jobId)*
Responsibilities: Cancels a pending or in-progress batch notification job, preventing further notifications from being sent.

Data ETL

- *DataSourceConfig registerDataSource(String sourceId, DataSourceType type, ConnectionConfig config, TransformationRules rules)*
Responsibilities: Registers a new external data source or updates an existing one. Configures connection parameters, authentication credentials, and normalisation transformation rules.
- *DataSourceConfig getDataSource(String sourceId)*
Responsibilities: Retrieves detailed configuration for a specific data source, including connection settings, transformation rules, and registration metadata.
- *List<DataSource> listDataSources(DataSourceType type, IngestionStatus statusFilter)*
Responsibilities: Retrieves all registered data sources, optionally filtered by type (TRAFFIC, TRANSIT, WEATHER, etc.) and current status (ACTIVE, STOPPED, ERROR).

- *void deleteDataSource(String sourceId)*
Responsibilities: Permanently removes a data source configuration from the system. Stops active ingestion if running, closes connections, and removes all configuration. Historical ingested data is retained for audit purposes.
- *void startIngestion(String sourceId, IngestionMode mode)*
Responsibilities: Initiates data extraction from the specified source in either STREAMING or BATCH mode, establishes connections, and begins the ETL pipeline.
- *void stopIngestion(String sourceId, String reason)*
Responsibilities: Halts data ingestion from a specific source, gracefully closes connections, flushes buffers, and logs the reason for stopping.
- *IngestionStatus getIngestionStatus(String sourceId)*
Responsibilities: Returns the current operational status of data ingestion for a source, including last successful pull timestamp, error counts, recent errors, data volume metrics, and connection health.
- *IngestionMetrics getIngestionMetrics(TimeRange range, List<String> sourceIds)*
Responsibilities: Returns aggregated metrics across specified sources and time range, including records processed, errors encountered, throughput rates, latency measurements, and data quality scores.

Observability

- *List<Country> listCountries()*
Responsibility: List Countries
- *List<City> listCities(String countryCode)*
Responsibility: List Cities based on the country code.

Traffic Data APIs

- *Stream<TrafficFlow> streamTrafficFlows(String cityId, LatLng northEast, LatLng southWest)*
Responsibility: Streams continuous, real-time updates on city-wide traffic flow.
- *ListTrafficFlowResponse listTrafficFlows(String cityId, Timestamp start, Timestamp end, LatLng northEast, LatLng southWest)*
Responsibility: Get city-wide traffic flow based on the bounding box and timestamp.
- *ListTrafficIncidentsResponse listTrafficIncidents(String cityId, String filter, Integer pageSize, Integer pageOffset, String orderBy, LatLng northEast, LatLng southWest)*
Responsibility: Retrieves a paginated list of traffic incidents (e.g., accidents, road closures) within a defined geographic bounding box, with filtering and ordering.

Public Transport APIs

- *ListVehiclesResponse listVehicles(String cityId, VehicleType vehicleType, LatLng northEast, LatLng southWest)*

Responsibility: List all vehicles in a city based on vehicle type (bus, tram, train).

- *Stream<VehicleLocations> streamVehicleLocations(String vehicleId)*
Responsibility: Streams real-time GPS locations for an active transport vehicle.
- *ListVehicleLocationsResponse listVehicleLocations(String vehicleId, Timestamp start, Timestamp end)*
Responsibility: Get a vehicle's location during a given time range.
- *GetVehicleStatusResponse getVehicleStatus(String vehicleId)*
Responsibility: Fetches the latest operational status and last reported location for a specific vehicle.

City Traffic Camera Feed

- *ListCamerasInAreaResponse listCamerasInArea(String cityId, LatLng northEast, LatLng southWest)*
Responsibility: Returns a list of available camera feeds within a geographic bounding box in a city.
- *Stream<CameraImages> streamCameraImages()*
Responsibility: Streams real-time images or video frames from key intersection traffic cameras.
- *GetCameraImageResponse getCameraImage(String cameraId, Timestamp timestamp)*
Responsibility: Retrieves the latest or a timestamped image from a specific traffic camera ID for visual verification.
- *GetCameraMetadataResponse getCameraMetadata(String cameraId)*
Responsibility: Fetches the location, direction, and operational status metadata for a specific camera.
- *ListCameraImageHistoryResponse listCameraImageHistory(String cameraId, Timestamp start, Timestamp end)*
Responsibility: Retrieves paginated historical image records for a specific camera over a time range.

PowerCheck API

- *Stream<PowerUsage> streamPowerUsage(String cityId)*
Responsibility: Streams continuous, real-time updates on power usage in a grid of the city grid.
- *ListPowerOutagesResponse listPowerOutages(String cityId, String filter, Integer pageSize, Integer pageOffset, String orderBy)*
Responsibility: Retrieves a paginated list of active power outages, filtered by severity and location, with ordering.
- *GetOutageDetailsResponse getOutageDetails(String outageId)*
Responsibility: Fetches detailed information for a specific power outage, including estimated restoration time and affected scale.

Meteorological Service API

- *GetCurrentWeatherResponse getCurrentWeather(LatLng location)*
Responsibility: Fetches the latest weather data (temp, wind, visibility, rainfall) for a specific geographic location.
- *ListRainfallIntensityResponse listRainfallIntensity(String filter, Integer pageSize, Integer pageOffset, String orderBy)*

Responsibility: Retrieves a paginated list of locations currently experiencing high rainfall intensity.

- *ListActiveWeatherWarningsResponse listActiveWeatherWarnings(String filter, WarningLevel level)*

Responsibility: Returns a filtered list of all current and active severe weather warnings.

- *GetAirQualityIndexResponse getAirQualityIndex(LatLng location, Timestamp time)*

Responsibility: Provides the air quality index for a location at a specific time, critical during fires or explosions.

Water Level Data

- *StreamRiverLevelsResponse streamRiverLevels()*

Responsibility: Streams real-time data from sensors on rivers and tidal information.

- *ListSensorReadingsResponse listSensorReadings(String filter, Integer pageSize, Integer pageOffset, String orderBy, WaterBodyType bodyType)*

Responsibility: Retrieves a paginated list of the latest sensor readings, filtered by river name or water body type, with ordering.

- *GetSensorStatusResponse GetSensorStatus(String sensorId)*

Responsibility: Fetches the current operational status and last reading time for a specific water level sensor.

- *ListFloodRiskAreasResponse listFloodRiskAreas(String filter, WaterLevelThreshold threshold)*

Responsibility: Returns a filtered list of areas currently at or exceeding a specific flood risk level.

- *ListAffectedRoadsResponse listAffectedRoads(String incidentId)*

Responsibility: Returns a list of roads currently confirmed as damaged or made impassable due to a water incident.

Hospital Network Data

- *ListHospitalStatusesResponse listHospitalStatuses(String filter, Integer pageSize, Integer pageOffset, String orderBy, HospitalType type)*

Responsibility: Retrieves a paginated list of hospital statuses, filtered by location, type, or operational capacity level, with ordering.

- *GetBedCountResponse getBedCount(String hospitalId, BedCategory category)*

Responsibility: Fetches the current number of available beds for a specific hospital, filtered by category (e.g., 'ICU', 'General').

- *ListNearHospitalsResponse listNearHospitals(LatLng location, Integer maxResults, String filter, BedCategory minAvailability)*

Responsibility: Returns a list of the nearest hospitals to a coordinate, ordered by distance and filtered by a minimum availability threshold.

- *GetCapacityRatingResponse getCapacityRating(String hospitalId)*

Responsibility: Provides a simple rating of the hospital's current overall capacity for quick decision-making.

- *Stream<AmbulanceLocations> streamAmbulanceLocations()*

Responsibility: Streams the live GPS coordinates and movement of all available and dispatched ambulances.

- *ListAmbulanceLocationsResponse listAmbulanceLocations(String filter, Integer pageSize, Integer pageOffset, String orderBy, AmbulanceStatus status)*
Responsibility: Retrieves a paginated list of ambulance locations, filtered by their current operational status, with ordering.
- *GetClosestAmbulanceResponse getClosestAmbulance(LatLng location, String filter)*
Responsibility: Identifies and returns the nearest available ambulance unit to a given coordinate, with optional filtering.
- *ListDispatchesResponse listDispatches(String filter, Integer pageSize, Integer pageOffset, String orderBy, PriorityLevel priority)*
Responsibility: Returns a paginated list of all active or recent dispatch calls, filtered by priority level.

Police/Fire Brigade Network

- *Stream<ServiceUnitLocations> streamServiceUnitLocations()*
Responsibility: Streams real-time GPS coordinates of active Police and Fire Brigade units.
- *ListServiceUnitsResponse listServiceUnits(String filter, Integer pageSize, Integer pageOffset, String orderBy, ServiceType type)*
Responsibility: Retrieves a paginated list of unit locations, filtered by service type and operational status, with ordering.
- *GetUnitCoordinationResponse getUnitCoordination(String incidentId)*
Responsibility: Returns the locations of all Police and Fire units currently assigned to a specific incident ID for coordination.
- *SearchAvailableResourcesResponse searchAvailableResources(LatLng location, ServiceType type, String resourceType)*
Responsibility: Searches for and returns the locations of available specific resources near a coordinate.
- *GetClosestUnitByTypeResponse getClosestUnitByType(LatLng location, ServiceType type)*
Responsibility: Identifies and returns the nearest active unit of a specified service type to a given point.

Social Media Monitoring API

- *StreamSocialAlertsResponse streamSocialAlerts()*
Responsibility: Streams real-time, filtered posts from social media and news sources related to disasters.
- *ListKeywordHitsResponse listKeywordHits(String filter, Integer pageSize, Integer pageOffset, String orderBy, KeywordSet set)*
Responsibility: Retrieves a paginated list of social media mentions, filtered by keyword set and location, with ordering.
- *ListUnconfirmedWarningsResponse listUnconfirmedWarnings(String filter, Integer pageSize, Integer pageOffset)*
Responsibility: Returns a paginated list of disaster warnings from social media that have not yet been correlated with official data.
- *SearchPostsByLocationResponse searchPostsByLocation(LatLng location, float radius, String filter)*
Responsibility: Searches for social media posts tagged or mentioning a specific location within a radius.

- *CorrelateAlertResponse correlateAlert(String socialAlertId, String officialDataId)*
Responsibilities: Allows an ERT member to manually correlate a social media alert with a confirmed, official incident.

Map

- *Coordinates geocode(String address)*
Responsibilities: Converts a text address into geographic coordinates
- *String reverseGeocode(Coordinates coords)*
Responsibilities: Converts geographic coordinates into a human-readable address
- *List<MapFeature> searchNearbyFeatures(Coordinates location, double radiusKm, List<FeatureType> types, String query)*
Responsibilities: Searches for features near a location within a radius, optionally filtered by types and query string. Covers shelters, incidents, units, etc.
- *MapFeature getFeatureAtLocation(Coordinates coords)*
Responsibilities: Returns feature details when the user clicks/taps on the map
- *List<RouteOption> getRouteOptions(Coordinates origin, Coordinates destination)*
Responsibilities: Retrieves alternative route options between two coordinates, considering live traffic, hazards, and evacuation constraints

Incident Registry

- *Incident createIncident(IncidentRequest request)*
Responsibilities: Creates a new incident record in the registry with all required metadata, including type, location, severity, description, and timestamp. Returns the created incident with a unique incident ID.
- *Incident getIncident(String incidentId)*
Responsibilities: Retrieves complete details of a specific incident by its unique ID, including current status, timeline, and all associated metadata.
- *ListIncidentResponse listActiveIncidents(IncidentFilter filter, Integer pageSize, Integer pageOffset)*
Responsibilities: Returns a paginated list of all currently active incidents, with optional filtering by type, severity, location, or time range.
- *Incident updateIncident(String incidentId, IncidentUpdateRequest update)*
Responsibilities: Updates an existing incident's details such as severity level, status, affected area, or description. Maintains version history for audit purposes.
- *void closeIncident(String incidentId, IncidentResolutionReport report)*
Responsibilities: Marks an incident as resolved or closed, records final status, completion time, and resolution details for historical records.
- *IncidentStatus getIncidentStatus(String incidentId)*
Responsibilities: Returns the current operational status of an incident (e.g., ACTIVE, CONTAINED, RESOLVED, ESCALATED) along with the last update timestamp.
- *List<Incident> listIncidents(LatLng center, Float radius, SeverityLevel severity, TimeRange range, IncidentFilter filter)*

Responsibilities Retrieves all incidents that match optional filters:
 Location-based filtering: When center and radius are provided, returns incidents within that geographic area.

Severity-based filtering: When severity is specified, returns incidents of that severity level.

Time-based filtering: When range is given, restricts results to incidents occurring within the specified time window.

Additional filters: The filter parameter allows applying other criteria (e.g., incident type, status, or source).

- *void linkDispatch(String incidentId, String dispatchId)*
 Responsibilities: Associates a dispatch operation with an incident, creating a linkage between the incident record and deployed emergency units.
- *void linkEvacuationPlan(String incidentId, String evacuationPlanId)*
 Responsibilities: Link an evacuation plan to an incident, ensuring coordinated tracking of evacuation operations related to the disaster.
- *void linkTrafficDetour(String incidentId, String detourId)*
 Responsibilities: Associate a traffic detour with an incident to maintain a complete picture of traffic management actions taken in response.
- *List<AssociatedResource> getIncidentResources(String incidentId)*
 Responsibilities: Returns all associated resources, dispatches, evacuation plans, and detours linked to a specific incident for a comprehensive operational view.
- *IncidentMetrics getIncidentMetrics(TimeRange range, MetricFilter filter)*
 Responsibilities: Provides aggregated analytics, including incident count by type, average resolution time, severity distribution, and response effectiveness metrics.
- *IncidentTimeline getIncidentTimeline(String incidentId)*
 Responsibilities: Returns a chronological timeline of all events, updates, and actions associated with an incident from creation to resolution.
- *ListIncidentResponse searchIncidents(SearchCriteria criteria, Integer pageSize, Integer pageOffset)*
 Responsibilities: Performs advanced search across incident records using multiple criteria, including location, type, severity, date range, and status.
- *ValidationResult validateIncident(IncidentRequest request)*
 Responsibilities: Validates incident data before creation, checking for duplicate reports, verifying location coordinates, and ensuring all required fields are present.
- *void escalateIncident(String incidentId, EscalationReason reason, SeverityLevel newSeverity)*
 Responsibilities: Escalates an incident to a higher severity level, triggers appropriate notifications, and logs the escalation reason for audit trails.
- *void publishIncidentEvent(String incidentId, IncidentEventType eventType, EventData data)*
 Responsibilities: Publishes incident-related events to the Event Bus for consumption by other services such as notification, mapping, and dispatching systems.

Communication Platform

- *MessageResponse sendMessage(String senderId, String receiverId, MessagePayload payload)*
Responsibilities: Sends a structured message between any two actors (e.g., ERT ↔ Field Unit, ERT ↔ Citizen). Supports text, image, and location payloads with message type metadata and delivery status tracking.
- *ListMessageResponse getConversationHistory(String conversationId, TimeRange range, Integer pageSize, Integer pageOffset)*
Responsibilities: Retrieves chronological message logs for a conversation, with pagination and optional time-based filtering. Used for audit, dashboard review, or post-incident reporting.
- *DeliveryStatus getMessageStatus(String messageId)*
Responsibilities: Returns the current delivery and acknowledgement status of a message (e.g., Sent, Delivered, Read). Used to ensure critical communications have reached intended recipients.
- *Boolean initiateVoiceChannel(String incidentId, List<String> participantIds)*
Responsibilities: Creates a secure, low-latency voice channel for coordination between multiple responders. Integrates with IAM for authentication and uses encrypted VoIP streaming.
- *Boolean endVoiceChannel(String channelId)*
Responsibilities: Terminates an active voice session, cleans up resources, and logs the communication summary for auditing.
- *void broadcastMessage(BroadcastRequest request)*
Responsibilities: Broadcasts urgent messages to multiple recipients (e.g., all field units within a geofence). Supports priority tiers (P0–P3) and integrates with the Notification function for multichannel delivery.
- *ListConversationFlagResponse getFlaggedConversations(TimeRange range, Integer pageSize, Integer pageOffset)*
Responsibilities: Retrieves conversations flagged for review due to distress, misinformation, or emergency escalation triggers. Used for moderation and compliance auditing.

Dispatching

- *Dispatch createDispatch(DispatchRequest request)*
Responsibilities: Creates a new dispatch assignment, selects the most appropriate available unit based on incident type, severity, and proximity, and initiates the dispatch process.
- *void assignUnitToIncident(String unitId, String incidentId, Route route)*
Responsibilities: Assigns a specific emergency unit to an incident and provides the calculated route to the disaster location.
- *void updateUnitStatus(String unitId, UnitStatus status, Coordinates currentLocation)*
Responsibilities: Updates the operational status and current location of an emergency unit (e.g., dispatched, en route, on-scene, available).
- *Dispatch getDispatchDetails(String dispatchId)*
Responsibilities: Retrieves complete details of a specific dispatch, including unit information, incident details, route, timestamps, and current status.
- *List<Dispatch> getActiveDispatches()*
Responsibilities: Returns all currently active dispatches across the system for monitoring and coordination purposes.

- *void notifyUnit(String unitId, DispatchNotification notification)*
Responsibilities: Send dispatch instructions, updates, or alerts to a specific emergency unit through the notification component.
- *void cancelDispatch(String dispatchId, String reason)*
Responsibilities: Cancels an active dispatch, returns the unit to available status, and logs the cancellation reason.
- *void completeDispatch(String dispatchId, DispatchReport report)*
Responsibilities: Marks a dispatch as completed, processes the final report from the unit, and updates system records.
- *ListUnitResponse getAllUnits(String filter, Integer pageSize, Integer pageOffset, String orderBy)*
Responsibilities: Returns the complete registry of all emergency units with their current status, type, location, capabilities, and availability.
- *DispatchMetrics getDispatchMetrics(TimeRange range)*
Responsibilities: Provides performance analytics, including average response times, unit utilisation rates, and dispatch success metrics.
- *void reassignDispatch(String dispatchId, String newUnitId, String reason)*
Responsibilities: Transfers an active dispatch from one unit to another, typically used when the original unit encounters issues or a closer unit becomes available.

Traffic Management

- *TrafficStatus getCurrentTrafficStatus(BoundingBox area)*
Responsibility: Returns live road status and congestion levels within the specified region.
- *DetourPlan generateDetour(RouteInput input, IncidentDetails incident)*
Responsibility: Creates detour recommendations based on road closures and real-time congestion.
- *void closeRoad(String roadId, String reason, Duration duration)*
Responsibility: Applies a road closure and records its operational details.
- *void openRoad(String roadId)*
Responsibility: Reopens a previously closed road and updates internal maps.
- *List<RoadClosure> listActiveClosures(BoundingBox area)*
Responsibility: Retrieves all current closures for the selected geographic region.
- *void publishTrafficAlert(TrafficAlert alert)*
Responsibility: Broadcasts a traffic alert to internal services and the notification platform.
- *TrafficMetrics getTrafficMetrics(TimeRange range)*
Responsibility: Provides aggregated traffic patterns and historical congestion intelligence.
- *Route calculateRoute(Coordinates start, Coordinates end, RouteOptions options)*
Responsibility: Computes an optimal route factoring in traffic, closures, and disaster constraints.
- *Route recalculateRoute(String dispatchId, Coordinates currentLocation)*
Responsibility: Recomputes a route for an active emergency dispatch based on live updates.
- *RoutePreview previewEvacuationRoute(EvacuationPlan plan)*

Responsibility: Generates a preview path for evacuation planning before final approval.

- *List<Route> getAlternativeRoutes(Coordinates start, Coordinates end)*
Responsibility: Provides multiple route candidates based on safety and travel efficiency.
- *CachedRoute getCachedRoute(String requestId)*
Responsibility: Returns a previously computed route if available to reduce recalculation time.
- *void publishRouteUpdate(String dispatchId, Route updatedRoute)*
Responsibility: Sends updated routing directions to dispatch and emergency units.
- *RoutingMetrics getRoutingPerformance(TimeRange range)*
Responsibility: Reports route computation time, cache hit rate, failure events, and recalculation frequency.

Evacuation Planning

- *EvacuationPlan generateEvacuationPlan(Incident incident, Constraints constraints)*
Responsibilities: Generates a new evacuation plan using incident data and system constraints (traffic, weather, transport).
- *EvacuationPlan getEvacuationPlan(String planId)*
Responsibilities: Retrieves a specific evacuation plan from storage for visualisation or update.
- *EvacuationPlan updateEvacuationPlan(String planId, UpdatedData data)*
Responsibilities: Modifies an existing evacuation plan based on evolving conditions.
- *void publishEvacuationPlan(String planId)*
Responsibilities: Send the finalised evacuation plan for visualisation, alerting, and operational execution.
- *List<EvacuationPlan> getActiveEvacuationPlans()*
Responsibilities: Returns all ongoing evacuation plans, useful for monitoring and dashboards.

Support System

- *SupportResponse sendMessage(String userId, String message)*
Responsibilities: Processes a user message, performs emotional-support NLP and intent detection, and returns an appropriate response with safety-checked content.
- *ConversationHistory getConversationHistory(String userId, TimeRange range)*
Responsibilities: Retrieves a user's conversation history for continuity, context, and compliance logging.
- *RiskAnalysis analyzeMessageRisk(String message)*
Responsibilities: Evaluates user messages for emotional distress, panic indicators, misinformation, and self-harm signals using predefined crisis-detection rules.

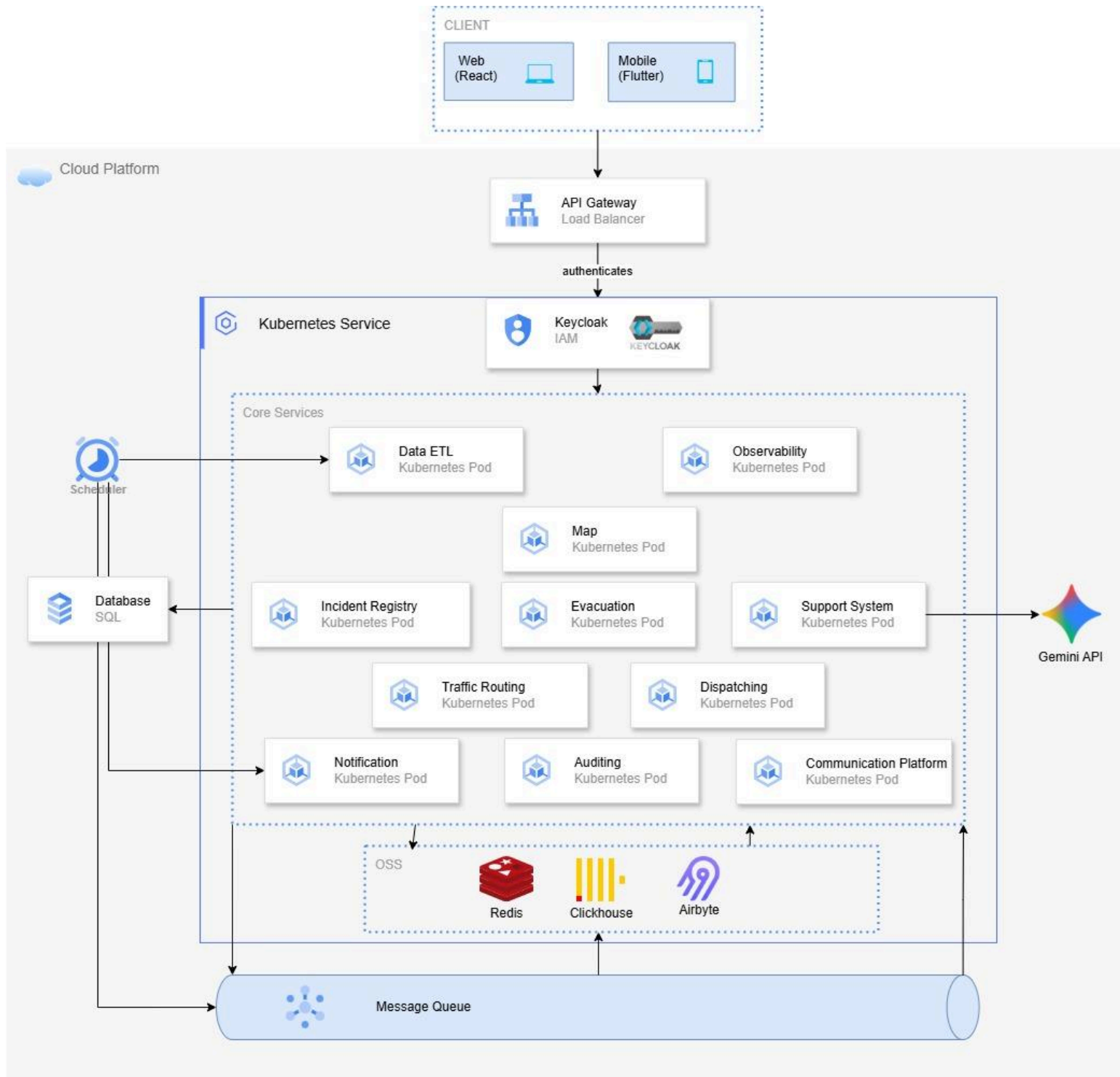
- *EscalationStatus escalateToHumanSupport(String userId, EscalationReason reason)*
Responsibilities: Escalates high-risk or distress cases to trained human support agents or mental-health responders based on severity rules.
- *void notifyEmergencyServices(String userId, Coordinates location, String severity)*
Responsibilities: Alerts emergency mental-health responders or crisis hotline services when threats to life or immediate self-harm indicators are detected.
- *Boolean requestWellbeingCheck(String userId)*
Responsibilities: Schedules a follow-up wellbeing check for a user who previously exhibited moderate risk behavior or reported emotional distress.
- *List<SelfHelpResource> listSelfHelpResources(String language, CrisisCategory category)*
Responsibilities: Returns curated mental-health guidance, coping exercises, government helpline links, and educational resources tailored to the crisis context and language.

Auditing

- *void logAuditEvent(String serviceId, String type, String timestamp, AuditPayload payload)*
Responsibilities: Asynchronously records a structured audit event, ensuring it is timestamped, validated, and persisted immutably in the central log store.
- *ListAuditRecordResponse queryAuditLog(String serviceId, TimeRange range, String payloadFilter, Integer pageNo, Integer pageOffset)*
Responsibilities: Queries the immutable log store based on provided filtering criteria (e.g., service ID, time range, payload filter, page number, and offset) to retrieve historical audit records.

2. Technical Architecture

2.1. Diagram



This system is designed as a cloud-native, microservices-based architecture to ensure high availability, scalability, and resilience for city-wide disaster response. It separates client-facing applications from backend services, which are containerized and managed within a central Kubernetes cluster. Communication between services is handled through a combination of synchronous API calls and an asynchronous message queue.

All client requests, originating from both the React-based Web application and the Flutter-based Mobile app, are directed to a unified API Gateway. This gateway acts as a load balancer and the primary security checkpoint, managing traffic routing to the appropriate backend services. It handles initial user authentication by interfacing directly with Keycloak (IAM), which runs as a dedicated service within the Kubernetes cluster.

The core of the platform is the Kubernetes Service, which orchestrates all backend microservices, each running in its own pod. This containerized approach allows for individual services, such as Incident Registry, Map, Traffic Routing, and Dispatching, to be scaled, updated, and deployed independently. Keycloak is integrated at this level, providing robust Identity and Access Management to secure inter-service communication and enforce role-based access control across the entire system.

Data ingestion is initiated by an external Scheduler, which triggers the Data ETL pod to fetch and process data. This processed information is then passed to the Observability service. Core incident data is managed by the Incident Registry service, which uses a primary SQL Database for persistent storage. This database, in turn, feeds events and data into a central Message Queue, creating an asynchronous backbone for the system.

This event-driven architecture allows for real-time, decoupled communication. Services like the Notification pod (which also reads from the SQL database) and the Communication Platform subscribe to the Message Queue to receive immediate updates, enabling them to alert users and facilitate communication without directly coupling to the services that produce the data. The Support System service also operates within this cluster, leveraging the external Gemini API to provide intelligent assistance.

A specialized Open Source Software (OSS) layer, consisting of Redis, Clickhouse, and Airbyte, provides high-performance data handling capabilities. Redis is likely used for in-memory caching to speed up responses, while Clickhouse serves as an analytical database for high-volume data. These tools support the core services by managing large-scale data processing, caching, and analytics, ensuring the system remains responsive during a large-scale emergency.

2.2. Quality of Service Technical Requirements

Security Requirements

<i>Security Requirement</i>	<i>Impact on the Project and how the Technical Architecture will address the impact</i>
Encryption - Do transactions need to be encrypted? - Level of encryption? (e.g., 40-bit encryption in the US)	All client-server and inter-service communication will be encrypted using HTTPS (TLS 1.3), the same encryption standard that secures banking websites. Data stored in databases or cloud storage is automatically encrypted by Google Cloud KMS. This ensures that even if data is intercepted, it cannot be read by unauthorized parties.
User Identification - uid/pw, cookies, certificates, application-level? - Existing customer database that should be used to identify online visitors?	The system uses Keycloak OAuth 2.0 / Google Sign-In for standard user authentication (login with Google or secure account credentials). Each internal service (e.g., Data Ingestion → Validator) uses service tokens or mutual TLS (mTLS) to verify identity. Every request is bound to a verified identity, either a human user or a trusted service.
Access to data - Do you need to restrict access to parts of the site? - What privacy rules should be applied to information provided by users	System Administrators have full access to view and edit all resources. Emergency Response Teams can access sensitive operational data. Public users can only view anonymized, safe data (such as city maps or alerts). Role-Based Access Control (RBAC) is enforced to determine access permissions per user role.
What are the legal requirements and policies for auditing content, changes and transactions?	Every critical action, such as data ingestion, dispatch operations, or evacuation plan approval, is recorded in an audit log (capturing who performed what action and when). These logs are stored securely in an immutable format (cannot be altered). This supports post-incident investigations and regulatory compliance audits.
Do you plan to use a secure demilitarised zone into which your project server code could be placed?	No.

System Management

<i>System Requirement</i>	<i>Management</i>	<i>Impact on the Project and how the Technical Architecture will address the impact</i>
Do you have access to the infrastructure required to install and run your own server?		Yes. The system is deployed on Google Cloud Platform (GCP) . Code is deployed using managed containers (Cloud Run / GKE), and data is stored in managed databases including PostgreSQL, MinIO, and ClickHouse.
What are the response time targets?		Response time requirements vary by operation priority: • <i>Critical</i> : < 1 second • <i>High</i> : < 2 seconds • <i>Medium</i> : < 3 seconds • <i>Low</i> : < 5 seconds
Availability: - What hours should the service be available? - Is it acceptable to have any scheduled downtime for maintenance? - How important is it that the service never be interrupted, even for unscheduled component failures? - If interruptions do occur, what should be the target time for resuming service?		The system is available 24/7 Scheduled maintenance is permitted with 24-hour advance notice and is limited to predefined maintenance windows. Critical functions (event bus, data ingestion, dispatching) are designed for high availability using multi-availability zone (multi-AZ) deployment. Recovery Time Objective (RTO): • Partial failures: < 1 hour • Major failures: < 4 hours RPO (data loss tolerance): • <i>Real-time event streams</i> (Pub/Sub replication): near-zero data loss • <i>Batch analytics</i>: up to 15 minutes of acceptable data loss
How should partial or total service failures be monitored and handled?		Each service sends health and performance data to Prometheus + Grafana dashboards. If a component fails (e.g., message lag or API errors), alerts are sent to the team via Slack/email. If a service crashes, another replica automatically takes over to maintain availability.
Do you need a recovery plan, or will it be covered by existing processes?		Yes, a dedicated recovery plan is in place. Backups, Pub/Sub message replay, and Airflow re-processing capabilities enable data and service recovery within 1 hour. Minor failures are automatically recovered through container restart mechanisms in Google Cloud.
Tracking/Documenting: How should the architecture support the process of		Each service sends logs and metrics to a centralized logging system (Prometheus + ELK Stack).

problem reporting, tracking and fixing? What statistics do you need to keep about the site, and how will they be analysed? What instrumentation should be included in the design to measure performance, response times and availability? Should the architecture include a repository for statistical data?	Alerts automatically create tickets (GitHub Issues / Jira) and notify the team. Failed messages are routed to a Dead Letter Queue (DLQ) for manual review and retry. All fixes and incidents are logged for auditing and continuous improvement. Metrics Tracked: • Data ingestion rate • Error counts and DLQ message volume • Consumer lag (Pub/Sub) • API latency • Notification delivery rates • Dispatch response times • User activity patterns Analysis Methods: • Real-time dashboards (Grafana) • Automated alerting rules • Periodic reports (daily/weekly) with anomaly detection Instrumentation: • Prometheus metrics exported from each service • Structured JSON logs • Distributed tracing (Jaeger) • Health check endpoints • DLQ metrics and retry counters Statistical Repository: • Time-series data and Long-term analytics stored in ClickHouse • Historical queries and reporting capabilities via ClickHouse
--	---

Client-side Management

<i>Client-side Management Requirement</i>	<i>Impact on the Project and how the Technical Architecture will address the impact</i>
Who is the customer? (Internet or Intranet) – affects browser choice	Emergency Response Teams (internal users) access the operator dashboard. Public users (external users) view maps and receive alerts.
What is the level of the user's skill?	Emergency Response Team members are trained professionals, so their interface can include technical details and advanced features. Public users are general citizens, so their interface is kept simple and visually intuitive.
What languages should the site support?	Multilingual

What are the user's usage patterns? (search or browse)	Emergency Response Teams use the map dashboard continuously for active monitoring. Public users primarily browse information or receive notifications.
How will the application maintain state?	Users authenticate via OAuth and receive a JWT token. This token maintains their authenticated session until expiration. Temporary user preferences are stored locally in browser storage.
Is there a need to distribute application code, and if so, how will it be done?	Container images are stored in a registry (Docker → GCR/GHCR) and deployed via Kubernetes (GKE) or Cloud Run. The frontend is delivered via a CDN. Deployment is automated through a CI/CD pipeline.
How will the choice of client affect the end-to-end response? (HTML, JavaScript, AJAX, JQuery, VBScript?)	The system uses a modern JavaScript (React) client with WebSocket communication. This approach reduces full-page reloads, transmits only incremental data, and enables live map updates. This improves end-to-end response time and enhances user experience.
What are the different user interfaces needed?	Operator Dashboard – for Emergency Response Team members to monitor disasters, approve evacuation plans, and dispatch units (real-time map view). Admin Console – for system administrators to manage data sources, users, and review DLQ/retry logs. Public Alert Portal – for citizens to view active alerts, evacuation routes, and updates. Support Chatbot Interface – for users to ask questions or request assistance during disasters.

Network Management

<i>Network Management Requirement</i>	<i>Impact on the Project and how the Technical Architecture will address the impact</i>
Will the solution involve the internet? What protocols will be used? (HTTP? HTTPS? FTP? RMI? Messaging? Etc)	Yes, the system operates over the internet using secure protocols: • HTTPS for API communication • WebSocket (TLS) for real-time updates • Pub/Sub messaging for reliable asynchronous communication
What about data, object and application placement? <i>projected transaction volumes, amount of data, interaction?</i>	Applications are deployed on cloud servers. Databases and object storage are hosted in distributed cloud infrastructure to handle moderate-to-high transaction volumes and concurrent user interactions efficiently.
What security functions are required/provided by chosen protocol? <i>Level of encryption will affect this, and also performance!</i>	TLS encryption ensures data confidentiality and integrity during transmission. Authentication and authorization mechanisms protect access to resources. Performance impact is minimal due to modern hardware acceleration for cryptographic operations.
How does the network affect end-to-end response time?	Network latency has a moderate effect on real-time updates. Optimized routing, caching strategies, and load balancing minimize delay and improve user experience.

Server-side Management

<i>Server-side Management Requirement</i>	<i>Impact on Project and how the Technical Architecture will address the impact</i>
Single server or multiple servers? Peer-to-Peer? Sensors?	Multiple cloud-based servers ensure scalability, redundancy, and load balancing. Sensors stream data through secure APIs.
Geographic location for servers?	Servers are deployed in geographically distributed data centers to ensure low latency and regional fault tolerance.
End-user client to server, or server to server also required?	Both are required: • Clients communicate with application servers for user-facing operations • Backend services exchange data server-to-server for real-time operational coordination
What security functions are required on the server?	Servers implement authentication, Role-Based Access Control (RBAC), encryption, and firewall protection to secure data and services.

How can the impact of the server on end-to-end response time be estimated, and catered for in the architecture?	Response time is optimized through load balancing, caching, and horizontal scaling. Monitoring tools track latency and performance metrics to identify bottlenecks.
---	---

Application Logic

<i>Server-side Management Requirement</i>	<i>Impact on the Project and how the Technical Architecture will address the impact</i>
Will the site use client-side executables? What are their connectivity requirements?	Yes, lightweight client-side components handle UI rendering and local input validation. These components require a stable internet connection for API communication with the server.
How will the application be split between client-side and server-side logic? (<i>affects communications for validation, etc/performance?</i>)	The client handles presentation logic and basic input validation. The server manages business logic, data processing, and storage. This distribution ensures efficient load balancing and faster response times.
Additional access security required?	Yes, security is enforced through: • Role-Based Access Control (RBAC) • Session management • Token-based authentication (JWT/OAuth)

Connectors

<i>Server-side Management Requirement</i>	<i>Impact on the Project and how the Technical Architecture will address the impact</i>
What external systems, applications and (sensor) data does your project need to access?	The system accesses sensor feeds, external APIs, and third-party services via secure REST and WebSocket connections.
How should data be transferred between different systems?	Data is exchanged through secure HTTPS and APIs. Large datasets or event-based data use asynchronous messaging (Pub/Sub).
How current does the information have to be? <i>Use caches?</i>	Real-time data is required for critical operations. Caching at API gateways and client applications ensures faster access and reduces server load.
Is synchronous or asynchronous access required? <i>Offline OK?</i>	Both are required: • Synchronous for request-response operations

	● Asynchronous for event streams and background processing Limited offline access is supported via local client-side caching.
Is access to different operating systems, network protocols, and application environments required? <i>Which connector? CICS? MQSeries? RPC?</i>	Yes, cross-platform compatibility is achieved via: ● REST APIs ● Message queues (Pub/Sub) These provide reliable inter-service communication across different environments.
Are additional security policies required?	Yes, strict security policies are enforced: ● API authentication (token-based) ● Encryption (TLS) ● Firewall rules for secure data transfer
Can scalability and performance requirements be predicted, and how will the project address these?	Auto-scaling, load balancing, and performance monitoring ensure system stability under variable workloads.